

第 1 章 LiteOS 简介

1.1 LiteOS 是什么？

Huawei LiteOS（后文用 LiteOS 表示 Huawei LiteOS）是华为公司在 2015 年发布的一个轻量级的面向物联网的操作系统，是华为物联网战略的重要组成部分，具备轻量级、低功耗、互联互通、组件丰富、快速开发等关键能力。最小内核尺寸仅 6KB。具备快速启动、低功耗等优势。LiteOS 具备零配置、自发现、自组网的特点，让 LiteOS 终端能够自动接入支持的网络，使硬件开发变得更加简单。并为终端厂商开发人员提供“一站式”完整软件开发平台，快速接入云，有效降低开发门槛、缩短开发周期。另外，华为在 2016 年 9 月发布了 LiteOS 开源版本，Kernel 源代码开源。在 2018 年 9 月发布了 LiteOSIDE 开发工具 LiteOS Studio V1.0。

1.2 LiteOS 的收费及使用要求

LiteOS 是一款“开源免费”的实时操作系统，面向 IoT（Internet of Things）领域轻量级物联网操作系统，遵循 BSD-3 开源许可协议，广泛应用于智能家居、个人穿戴、车联网、城市公共服务、制造业等领域，大幅降低设备布置及维护成本，有效降低开发门槛、缩短开发周期。同时提供端云协同能力，集成了 LwM2M、CoAP、mbed TLS、LwIP 全套 IoT 互联协议栈，且在 LwM2M 的基础上，提供了 AgentTiny 模块，用户只需关注自身的应用，而不必关注 LwM2M 实现细节，直接使用 AgentTiny 封装的接口即可简单快速实现与云平台安全可靠的连接。

LiteOS 自开源社区发布以来，围绕 NB-IoT（Narrow Band Internet of Things）物联网市场从技术、生态、解决方案、商用支持等多维度使能合作伙伴，构建开源的物联网生态，目前已经聚合了 50 多个 MCU 和解决方案合作伙伴，共同推出一批开源开发套件和行业解决方案，帮助众多行业客户快速的推出物联网终端

和服务，客户涵盖抄表、停车、路灯、环保、共享单车、物流等众多行业，为开发者提供“一站式”完整软件平台，有效降低开发门槛、缩短开发周期。

这里说到的开源，指的是读者可以免费得获取到 LiteOS 的源代码，读者的产品的全部代码都可以闭源，不用开源，免费的意思是无论读者是个人还是公司，都可以免费地使用，不需要掏一分钱。

对于 BSD-3（Berkeley Software Distribution）开源许可协议，使用者基本上可以“为所欲为”，可以自由的使用，修改源代码，也可以将修改后的代码作为开源或者专有软件再发布。但“为所欲为”的前提是当使用者发布使用了 BSD-3 协议的代码，或者以 BSD 协议代码为基础做二次开发自己的产品时，需要满足三个条件：

- （1）如果再发布的产品中包含源代码，则在源代码中必须带有原来代码中的 BSD 协议；

- （2）如果再发布的只是二进制类库/软件，则需要在类库/软件的文档和版权声明中包含原来代码中的 BSD 协议；

- （3）不可以用开源代码的作者/机构名字和原来产品的名字做市场推广。

1.3 LiteOS 的意义

目前，在 RTOS（Real Time Operating System）领域，我们能接触到的操作系统基本都来自国外，很少见到有国内的厂家在 RTOS 领域崭露头角。从早年最火的 μ COS，到如今市场占有率最高的 FreeRTOS，到获得安全验证最多的 RTX，再到盈利能力最强的 ThreadX，均来自国外，而华为作为全球顶尖的通讯科技公司，其开发的 LiteOS 作为一款国产的物联网操作系统，简单易用，低功耗设计，组件丰富等特性也将让 LiteOS 大放异彩。

1.4 为什么要学习 RTOS

当我们进入微控制器设计这个领域的时候，往往首先接触的都是裸机编程，即不加入任何 RTOS 的程序，所有的程序基本都是自己写的，所有的操作都是在一个无限的大循环里面实现。现实生活中的很多中小型的电子产品采用的都是裸

机编程，而且也能够满足需求。那为什么还要学习 RTOS 编程呢？主要的原因是随着微控制器性能的大幅提升，以及产品要实现的功能越来越多，原来基于裸机编程的系统已经不能够完美地解决问题，而且会使编程变得更加复杂，如果想提高系统性能，降低编程的难度，可以考虑引入 RTOS 实现多任务管理，这是使用 RTOS 的最大优势。

1.5 如何学习 RTOS

裸机编程和 RTOS 编程的风格有些不一样，而且有很多人说 RTOS 的学习很难，这就导致学习的人一听到 RTOS 编程就产生畏惧心理。那么到底如何学习一个 RTOS？最简单的就是在别人移植好的系统之上，看看 RTOS 里面的 API 使用说明，然后调用这些 API 实现自己想要的功能即可。完全不用关心底层的移植，这是最简单快速的入门方法。这种方法各有利弊，如果是做产品，好处是可以快速的实现功能，将产品推向市场，赢得先机，弊端是当程序出现问题的时候，因对这个 RTOS 不够了解，会导致调试困难，焦头烂额，一筹莫展。如果要真正掌握 RTOS，只会简单的调用 API 是不够的。

目前市场上现有的 RTOS，它们的内核实现方式都差不多，因此只需要深入学习其中一款就行。万变不离其宗，以后换到其他型号的 RTOS，使用起来也是得心应手。那如何深入的学习一款 RTOS？这里有一个最有效也是最难的方法，就是阅读 RTOS 的源码，深究内核和每个组件的实现方式，这个过程枯燥且痛苦。

由于课程学时的限制，本资料基于授课内容和相关案例，结合 RTOS 源码和 API 使用，让读者初步掌握 RTOS 原理和使用方法。

第 2 章 LiteOS 源码

2.1 LiteOS 源码简介

华为 LiteOS 的源码有两份，分别是 master 版本和 develop 版本，这两份 LiteOS 源码的核心是一样的。develop 版本提供了更强大的组件及丰富的例程，如 Wi-Fi 模块、NB-IoT 模块和 GPRS 模块，并且更新比 master 版本更快。使用 LiteOS 接入云端更加简便，而 master 版本发布则是改动较大的。本资料主要以 LiteOS 的内核为主，如 LiteOS 的任务等基础功能、IPC 通信机制、内存管理等，无论是使用 develop 版本还是 master 版本，其内核源码的实现方式都是一样的，华为官方建议下载 master 版本使用。

LiteOS 的源码可从 LiteOS GitHub 仓库地址 <https://github.com/LiteOS/LiteOS> 下载到，读者在移植时并不需要把整个 LiteOS 源码放进工程文件中，否则工程的代码量太大。只需把 LiteOS 源码中的核心部分单独提取出来，移植到自己的工程文件中即可。LiteOS 的源码文件夹结构如图 2.1 所示。可以看见有 8 个文件夹和一些文件。

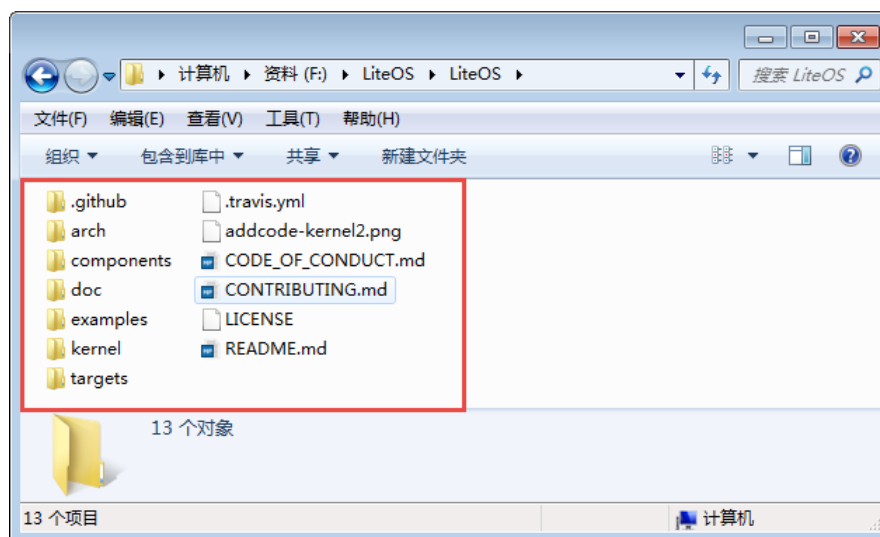


图 2.1 LiteOS 源码

2.2 LiteOS 源码核心文件夹

LiteOS 源码文件的 8 个文件夹中，arch、cmsis 和 kernel 为核心文件夹，移植时可将核心文件夹提取出来，添加到裸机工程根目录下的文件夹中，因为工程只需要有用的源码文件，而不是全部的 LiteOS 源码，所以可以避免工程过于庞大。LiteOS 源码核心文件夹的主要内容如表 2.1 所示。

表 2.1 LiteOS 核心文件夹的主要内容组成

文件夹	文件夹	文件夹	描述
arch	arm	arm-m	M 核中断、调度、Tick 相关代码
		common	arm 核公用的 cmsis core 接口
components	cmsis		LiteOS 提供的 cmsis os 接口实现
kernel	base	core	LiteOS 基础内核代码文件，包括队列、task 调度、软 timer、时间片等功能
		om	错误处理的相关文件
		include	LiteOS 内核内部使用的头文件
		ipc	LiteOS 中 IPC 通信相关的代码文件，包括事件、信号量、消息队列、互斥锁等
		mem	LiteOS 中的内核内存管理的相关代码
		misc	内存对齐功能及毫秒级休眠 sleep 功能
	include		LiteOS 开源内核头文件
	extenden	ticless to tickless	低功耗框架代码

这些文件夹里面的文件是 LiteOS 源码的核心文件，核心文件容量很小，可以直接将 LiteOS 核心文件夹下的所有文件夹复制到 STM32 裸机工程里面，让整个 LiteOS 跟随工程一起发布。使用这种方法打包的 LiteOS 工程，复制到任何一台电脑上面都是可以使用的，而不会提示找不到 LiteOS 的源文件，如下图所示。

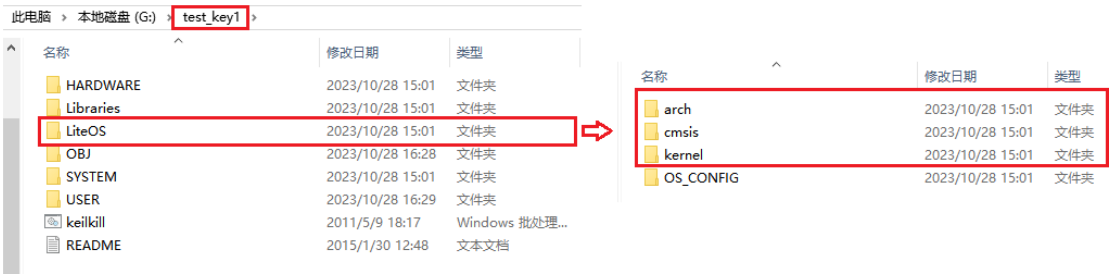


图 2.2 复制 LiteOS 核心文件夹到裸机工程

2.3 LiteOS 文件夹内容

本节对 LiteOS 完整源码文件夹中的内容作一个简单介绍。

(1) targets 文件夹

targets 文件夹里面存放的是板级工程代码（含原厂芯片驱动）， LiteOS 已经为各半导体厂商的评估板写好程序，这些程序就存放在 targets 文件夹下。本资料中的 LiteOS 版本是 master 版本，只有几款开发板的程序，如图 2.3 所示。



图 2.3 targets 文件夹内容

targets 文件夹中的每一个工程文件里都有具体的 LiteOS 系统初始化文件、配置文件等。例如，STM32F103RB_NUCLEO 工程文件夹中的 OS_CONFIG 是 LiteOS 功能的配置文件夹，里面的配置文件定义了很多宏，通过这些宏定义，用户可以根据需要裁剪 LiteOS 的功能。用户在使用 LiteOS 时，只需修改 OS_CONFIG 文件夹中的内容，其他文件并不需要改动。为了减小工程的大小，只需把 OS_CONFIG 文件夹提取出来即可，如图 2.4 所示。



图 2.4 提取 STM32F103RB_NUCLEO 文件夹中的 OS_CONFIG 文件夹内容

OS_CONFIG 文件夹中的 target_config.h 可用于配置裁剪与配置 LiteOS 的功能。如图 2.5 所示。

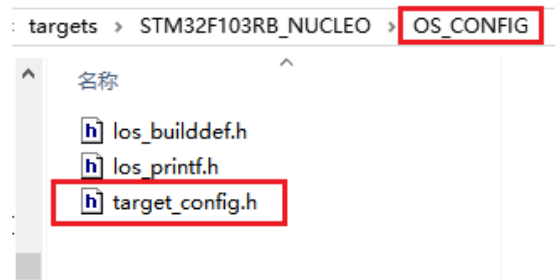


图 2.5 target_config.h 文件

(2) components 文件夹

在 LiteOS 中,除了内核外的其他第三方软件都是组件,如 agent_tiny、lwmm2m、lwip 和 mbedtls 等。这些组件就存放在 components 文件夹中。

(3) examples 文件夹

examples 目录中存放的是供开发者测试 LiteOS 内核的 demo 示例,此目录存放的是内核功能测试相关例程的代码。

(4) arch 文件夹

LiteOS 是软件,单片机是硬件,为了使 LiteOS 运行在单片机上面,LiteOS 和单片机必须关联在一起,那么如何关联呢?还是要通过代码来关联,这部分关联的文件叫接口文件,通常由汇编语言和 C 语言联合编写。这些接口文件都是跟硬件密切相关的,不同的硬件接口文件是不一样的,但都大同小异。编写这些接口文件的过程叫作移植,移植的过程通常由 LiteOS 和 mcu 原厂的人来负责,移植好的这些接口文件存放在 arch 文件夹的目录中。 LiteOS 在 arch\arm\arm-m 目录中存放了 cortex-m0、m3、m4 和 m7 内核的单片机的接口文件,使用了这些内核的 mcu 都可以使用里面的接口文件。通常网络上出现的大部分“移植某某 RTOS 到某某 MCU”的教程,其实准确来说,不能够用“移植”这个词语,应该用“使用 RTOS 官方的移植”这种表述方法。因为这些跟硬件相关的接口文件,大多数情况下 RTOS 官方都已经写好了,而用户只是使用而已。本资料中提到的“移植”也是“使用 LiteOS 官方的移植”意思,一般情况下这些底层的移植文件暂时无需深入理解,直接使用即可。

(5) kernel 文件夹

kernel 文件夹中存放的是 LiteOS 内核的源文件,是 LiteOS 内核的核心。前文已经简述了 kernel 文件夹的作用,此处就不再重复赘述。

2.4 target_config.h 文件相关内容讲解

前面提到 OS_CONFIG 文件夹中的 target_config.h 可用于配置裁剪与配置 LiteOS 的功能，移植时可以直接从 LiteOS 官方的工程文件夹中复制添加到裸机工程文件中，如图 2.6 所示。

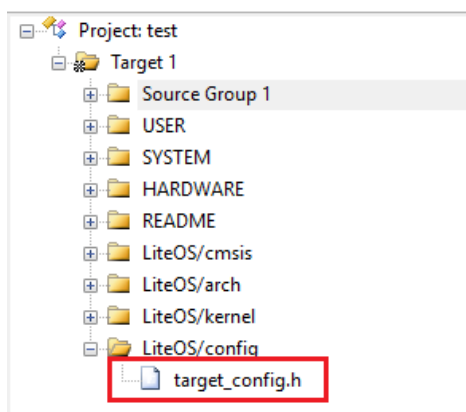


图 2.6 target_config.h 文件的移植

下面简单介绍 target_config.h 文件中与本资料内容相关的部分宏定义的含义，在对这些宏定义功能有所掌握的前提下，实际应用中可以进行修改以满足应用的需要。

(1) #define OS_SYS_CLOCK (SystemCoreClock)

OS_SYS_CLOCK 是配置 LiteOS 的时钟为系统时钟的参数，在 STM32 实验板上系统时钟为 SystemCoreClock= SYSCLK_FREQ_72MHz，也就是 72MHz。

(2) #define LOSCFG_BASE_CORE_TICK_PER_SECOND (1000UL)

LOSCFG_BASE_CORE_TICK_PER_SECOND 表示操作系统每秒钟产生 Tick 的数量，Tick 是指操作系统时钟节拍的周期。系统时钟节拍就是系统以固定的频率产生中断（时基中断），并在中断中处理与时间相关的事件，推动所有任务向前运行。时钟节拍需要依赖于硬件定时器，在 STM32 裸机程序中经常使用的 SysTick 时钟是 CM3 的内核定时器，通常使用该定时器产生操作系统的时钟节拍。因此裸机程序中经常用于延时的 delay_us() 和 delay_ms() 在 LiteOS 程序中不能随意使用，因为这两个子函数均是利用 SysTick 实现延时，在 LiteOS 程序中使用会出现冲突。LiteOS 系统延时和阻塞时间都是以 Tick 为单位的，配置 LOSCFG_BASE_CORE_TICK_PER_SECOND 的值可以改变 SysTick 中断的频率，从而间接改变 LiteOS 的时钟节拍周期 ($T=1/f$)。如果将 LOSCFG_BASE_CORE

_TICK_PER_SECOND 的值设置为 1000，那么 LiteOS 的系统时钟节拍周期为 1ms。过高的系统节拍中断频率意味着 LiteOS 内核将占用更多的 CPU 时间，因此会降低效率，一般将 LOSCFG_BASE_CORE_TICK_PER_SECOND 的值设置为 50~1000。

(3) #define LOSCFG_PLATFORM_HWI NO

LOSCFG_PLATFORM_HWI 是硬件中断定制配置参数，YES 表明 LiteOS 接管了外部中断，一般建议设置为 NO，即不接管中断。

(4) #define LOSCFG_BASE_CORE_TSK_DEFAULT_PRIO 10

LOSCFG_BASE_CORE_TSK_DEFAULT_PRIO 这个宏定义表示默认的任务优先级，默认为 10，优先级数值越小表示任务优先级越高。

(5) #define LOSCFG_BASE_CORE_TSK_LIMIT 15

LOSCFG_BASE_CORE_TSK_LIMIT 这个宏定义表示 LiteOS 支持的最大任务个数（除去空闲任务），默认为 15。

(6) #define LOSCFG_BASE_CORE_TSK_IDLE_STACK_SIZE (0x500U)

LOSCFG_BASE_CORE_TSK_IDLE_STACK_SIZE 这个宏定义表示空闲任务的栈大小，默认为 0x500U 字节。

(6) #define LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE (0x2D0U)

LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE 宏定义表示默认的任务栈大小为 0x2D0U 字节，在任务创建的时候一般都会指定任务栈的大小，以适配不一样的应用任务，而如果没有指定则使用默认值。

(7) #define LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE (0x130U)

LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE 这个宏定义则表示任务最小需要的栈大小，栈大小应该是一个合理的值，如果太大，可能会导致内存耗尽，最小的栈大小默认为 0x130U。任务栈大小必须在 8 个字节的边界上对齐。该值的大小取决于它是否足够大以避免任务栈溢出。

(8) #define LOSCFG_BASE_CORE_TIMESLICE_TIMEOUT 10

LOSCFG_BASE_CORE_TIMESLICE_TIMEOUT 这个宏定义表示具有相同优先级的任务的最长执行时间，单位为系统节拍时钟周期，默认配置为 10。

(9) #define LOS_TASK_PRIORITY_HIGHEST 0

LOS_TASK_PRIORITY_HIGHEST 这个宏定义表示定义可用的任务的最高优先级。在 LiteOS 中默认最高优先级为 0，优先级数值越小，优先级越高。

(10) #define LOS_TASK_PRIORITY_LOWEST 31

LOS_TASK_PRIORITY_LOWEST 这个宏定义表示定义可用的任务最低优先级，在 LiteOS 中默认为 31，LiteOS 最大支持 32 个抢占优先级，优先级数值越大，优先级越低。

(11) #define LOSCFG_BASE_IPC_SEM YES

LOSCFG_BASE_IPC_SEM 这个宏定义表示信号量的配置项，配置为 YES 则表示默认使用信号量。

(12) #define LOSCFG_BASE_IPC_SEM_LIMIT 20

LOSCFG_BASE_IPC_SEM_LIMIT 这个宏定义表示 LiteOS 最大支持信号量的个数，默认为 20 个，用户可以自定义设置信号量个数。

(13) #define LOSCFG_BASE_IPC_MUX YES

LOSCFG_BASE_IPC_MUX 这个宏定义表示互斥锁的配置项，配置为 YES 则表示默认使用互斥锁。

(14) #define LOSCFG_BASE_IPC_MUX_LIMIT 15

LOSCFG_BASE_IPC_MUX_LIMIT 这个宏定义表示 LiteOS 最大支持互斥锁的个数，默认为 15。

(15) #define LOSCFG_BASE_IPC_QUEUE YES

LOSCFG_BASE_IPC_QUEUE 这个宏定义表示消息队列的配置项，配置为 YES 则表示默认使用消息队列。

(16) #define LOSCFG_BASE_IPC_QUEUE_LIMIT 10

LOSCFG_BASE_IPC_QUEUE_LIMIT 这个宏定义表示 LiteOS 最大支持消息队列量的个数，默认为 10。

(17) #define OS_SYS_MEM_SIZE

((UINT32)(_LOS_HEAP_ADDR_END_ - _LOS_HEAP_ADDR_START_ +1))

OS_SYS_MEM_SIZE 这个宏定义是系统的内存大小，大小为结束地址-起始地址+1。

第 3 章 创建任务

上一章讲解了 LiteOS 部分源码的内容，本章将从最简单的创建任务开始，讲解使用 LiteOS 实现 LED 翻转的功能，让读者能够对 LiteOS 的任务以及任务的创建有初步的了解。

3.1 硬件初始化

在启动 LiteOS 之前，应该先将系统需要的硬件初始化完毕。本章创建的任务需要用到开发板搭载的 LED，因此首先需要将 LED 硬件部分初始化，具体的处理是在 main.c 文件的 BSP_Init()函数中将 LED 初始化，如代码清单 3.1 加粗部分所示。

代码清单 3.1 BSP_Init()中添加硬件初始化函数

```
1  /*****
2  * @ 函数名   :   BSP_Init
3  * @ 功能说明:   板级外设初始化，所有开发板上的初始化均可放在这个函数里面
4  * @ 参数     :
5  * @ 返回值   :   无
6  *****/
7  static void BSP_Init(void)
8  {
9      /* STM32中断优先级分组为 4，即 4bit都用来表示抢占优先级，取值为 0~15
10       * 优先级分组只需要分组一次即可，以后如果有其他的任务需要用到中断，
11       * 都统一用这个优先级分组，千万不要再分组，切忌。*/
12
13
14     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
15
16     /* LED 初始化 */
17     LED_GPIO_Config();
18 }
```

在 main 函数中调用 BSP_Init()，当代码执行到 BSP_Init()函数时，操作系统的内容完全未涉及到，即 BSP_Init()函数所做的处理与裸机工程里面的硬件初始化是一样的。当执行完 BSP_Init()函数后，CPU 才会启动操作系统去运行用户创建完成的任务。

3.2 创建单任务

在本小节中将会创建一个任务，任务使用的任务栈和任务控制块的内存空间是在创建任务的时候由 LiteOS 动态分配的。在 LiteOS 系统中，每一个任务都是独立的，它们的运行环境都单独保存在自己的任务栈中，任务栈占用的是 MCU 内部的 SRAM（内存），创建的任务越多，需要使用的任务栈内存空间就越大，需要使用的 SRAM 也就越多，一个 MCU 能够支持创建多少个任务，取决于它的 SRAM 空间有多少。

3.2.1 动态内存空间从哪里来

任务栈和任务控制块的内存空间都是从 SRAM 分配的，具体分配到哪个地址由编译器决定。LiteOS 支持动态内存分配，任务栈和任务控制块的内存空间均由 LiteOS 分配，在 LiteOS 中定义了一个 `OS_SYS_MEM_SIZE` 宏定义表示系统可以管理的内存空间大小，这个宏定义在 `target_config.h` 中由用户配置（对于 STM32F1 系列默认的配置在 20kbyte 以内）。在创建任务的时候，系统从该内存空间分配需要的内存。

3.2.2 LiteOS 核心初始化

在开始创建任务之前，需要将 LiteOS 的核心组件进行初始化，LiteOS 提供了一个核心初始化函数——`LOS_KernelInit()`，它能够将整个 LiteOS 系统的核心部分初始化，在初始化完成后，读者即可根据自己需要创建任务。

核心初始化函数 `LOS_KernelInit()` 主要完成了以下工作：

（1）系统内存的初始化，LiteOS 所管理的内存只是一块内存区域，LiteOS 将它管理的内存初始化一遍，目的是为了后续能正常管理。

（2）如果系统接管中断，那么 LiteOS 会将所有的中断入口函数通过一个指针数组存储。而系统不接管中断则不会对中断入口函数进行处理，读者可以通过 `target_config.h` 文件中的 `LOSCFG_PLATFORM_HWI` 宏定义配置为是否（YES or NO）由系统接管中断。

（3）任务基本的底层初始化，如 LiteOS 采用一块内存来管理所有的任务控制块信息，系统最大支持 `LOSCFG_BASE_CORE_TSK_LIMIT`（15）+1 个任务

（包括空闲任务），该宏定义的值在 `target_config.h` 文件中配置，用户可以根据系统需求进行裁剪，以减少系统的内存开销。核心初始化函数会初始化系统必要的控制块链表等。

（4）如果用户使能了任务监视功能，那么初始化函数也会初始化对应的监视功能。

（5）如果用户使能了信号量、互斥锁、消息队列等 IPC 通信机制，那么在系统运行前也会将这些内核资源初始化，系统支持最大的信号量、互斥锁、消息队列个数在 `target_config.h` 文件中定义，可以由用户配置，当不需要那么多数量的时候可以进行裁剪，以减少系统的内存开销。

（6）如果系统使用了软件定时器，就必须使用消息队列（因为软件定时器依赖消息队列进行管理），同时会初始化相关的功能，除此之外系统还会创建一个软件定时器任务。

（7）LiteOS 会创建一个空闲任务，空闲任务在系统中是必须存在的，因为处理器是一直在运行的，当整个系统都没有就绪任务的时候，系统必须保证有一个任务在运行。空闲任务就是为这个目的而设计的，空闲任务的优先级最低，当用户创建的任务处于就绪态时，它可以抢占空闲任务的 CPU 使用权，从而执行用户创建的任务。空闲任务默认的任务栈大小，由 `target_config.h` 文件中的 `LOSCFG_BASE_CORE_TSK_IDLE_STACK_SIZE`（`0x500U`）指定，用户可以进行修改。

3.2.3 定义任务函数

在创建任务时，需要指定任务函数（或者称之为任务主体），它应该是一个无限的死循环，不能返回，如代码清单 3.2 所示。

代码清单 3.2 定义任务函数

```
1 /*****
2 * 函数名:   Test1_Task
3 * 功能:     Test1_Task任务实现
4 *****/
5
6 static void Test1_Task(void)
7 {
8     /* 任务都是一个无限循环，不能返回 */
9     while(1) {
10         LED0 = !LED0;
11         LOS_TaskDelay(1000); /*延时 1000个tick*/
12     }
13 }
```

代码清单 3.2(1): 每个独立的任务都是一个无限循环, 不能返回。

代码清单 3.2(2): 翻转 LED 灯。

代码清单 3.2(3): 调用系统延时函数, 保证任务得以进行切换, 延时 1000 个 Tick。

3.2.4 定义任务 ID 变量

在 LiteOS 中, 任务 ID (也可以理解为是任务句柄, 下文均采用任务 ID 表示) 是非常重要的, 因为它是任务的唯一标识, 任务 ID 本质是一个从 0 开始的整数, 是任务身份的标志 (读者也可以简单理解为任务的索引号)。在任务创建成功后, 系统会返回一个任务 ID 给用户, 用户可以通过任务 ID 对任务进行挂起、恢复、查询信息等操作, 在这之前需要用户定义一个任务 ID 变量, 用来存储返回的任务 ID。其定义如代码清单 3.3 所示。

代码清单 3.3 定义任务 ID 变量

```
1 /* 定义任务 ID 变量 */  
2 UINT32 Test1_Task_Handle;
```

3.2.5 任务控制块

每一个任务都含有一个任务控制块(TCB)。TCB 包含了任务栈指针 (stack pointer)、任务状态、任务优先级、任务 ID、任务名、任务栈大小等信息, 还可以反映出每个任务的运行情况, 任务控制块的内容如代码清单 3.4 所示。

任务入口函数: 每个新任务得到调度后将执行的函数。该函数由用户实现, 在任务创建时, 通过任务创建结构体指定。

任务优先级: 优先级表示任务执行的优先顺序。任务的优先级决定了在发生任务切换时即将要执行的任务, 处于就绪列表中最高优先级的任务将得到执行。

任务栈: 每一个任务都拥有一个独立的栈空间, 也称之为任务栈。任务栈保存的信息包含局部变量、寄存器、函数参数、函数返回地址等。发生任务切换时会将切出任务的上下文信息保存在任务自身的任务栈中, 在任务恢复运行时还原现场, 从而保证在任务恢复后不丢失数据, 继续执行。

任务上下文：任务在运行过程中使用到的一些资源，如寄存器等，称为任务上下文。当某个任务挂起时，系统中的其他任务得到运行，在任务切换的时候如果没有把切出任务的上下文信息保存下来，就会导致未知的错误。因此 LiteOS 在任务挂起的时候会将切出任务的上下文信息保存在任务栈中，在任务恢复的时候，系统将从任务栈中恢复挂起时的上下文信息，任务将继续运行。

代码清单 3.4 任务控制块 (los_tack.ph)

```
1 typedef struct tagTaskCB {
2     VOID                *pStackPointer; /*任务栈指针*/
3     UINT16              usTaskStatus;   /*任务状态*/
4     UINT16              usPriority;      /*任务优先级*/
5     UINT32              uwStackSize;     /*任务栈大小*/
6     UINT32              uwTopOfStack;    /*任务栈顶*/
7     UINT32              uwTaskID;        /*任务 ID*/
8     TSK_ENTRY_FUNC      pfnTaskEntry;    /*任务入口函数*/
9     VOID                *pTaskSem;       /*任务阻塞在哪个信号量*/
10    VOID                *pTaskMux;       /*任务阻塞在哪个互斥锁*/
11    UINT32              uwArg;            /*参数*/
12    CHAR                *pcTaskName;      /*任务名字*/
13    LOS_DL_LIST          stPendList;      /*挂起列表*/
14    LOS_DL_LIST          stTimerList;     /*时间相关列表*/
15    UINT32              uwIdxRollNum;
16    EVENT_CB_S          uwEvent;          /*事件*/
17    UINT32              uwEventMask;      /*事件掩码*/
18    UINT32              uwEventMode;      /*事件模式*/
19    VOID                *puwMsg;          /*内存分配给队列*/
20 } LOS_TASK_CB;
```

3.2.6 创建任务

创建任务的时候， 使用 LOS_TaskCreate()函数来创建一个任务，从前面的章节中读者 已经了解到每个任务的具体配置是需要用户定义的，不同的任务之间参数是不一样的，如代码清单 3.5 所示。

代码清单 3.5 创建任务

```
1 /*****
2  * 函数:   Creat_Test1_Task
3  * 功能:   创建Test1_Task任务
4  *****/
5 static UINT32 Creat_Test1_Task()
6 {
7     UINT32 uwRet = LOS_OK; //定义一个创建任务的返回类型， 默认为创建成功的返回值
```

```

8     TSK_INIT_PARAM_S task_init_param; /*定义一个局部变量 */ (1)
9
10    task_init_param.usTaskPrio = 5; /* 任务优先级, 数值越小, 优先级越高*/ (2)
11    task_init_param.pcName = "Test1_Task"; /* 任务名称 */ (3)
12    task_init_param.pfnTaskEntry = (TSK_ENTRY_FUNC)Test1_Task; (4)
13    task_init_param.uwStackSize = 0x1000; /* 任务栈大小 */ (5)
14
15    uwRet = LOS_TaskCreate (&Test1_Task_Handle, &task_init_param); (6)
16    return uwRet; (7)
17 }

```

代码清单 3.5 (1): 定义一个局部的任务参数结构体变量, 用于配置任务的参数, 如任务优先级、任务入口函数、任务名称、任务栈大小等信息。

代码清单 3.5 (2): 任务的优先级。优先级范围由 `target_config.h` 中的宏定义决定, 其中最高优先级为 `LOS_TASK_PRIORITY_HIGHEST` (默认为 0), 最低优先级为 `LOS_TASK_PRIORITY_LOWEST` (默认为 31)。在 LiteOS 中, 优先级数值越小, 任务优先级越高, 0 代表最高优先级。

代码清单 3.5 (3): 任务名字, 字符串形式。使用字符串的目的有两个: 一是方便用户调试; 二是因为 LiteOS 创建任务时不会给“任务名”分配内存, 而是直接使用用户传入的字符串, 使用字符串的方式 (C 语言里面以双引号包含的字符串) 编译器会在 `text` 段 (即 `flash`) 创建字符串实体。这样使用更安全, 如果在局部使用字符数组则可能会导致后续访问“任务名”时结果不可预知造成错误。

代码清单 3.5 (4): 任务入口函数, 即任务的具体实现函数, 一般来说任务函数是不允许退出的, 否则任务将通过 LR 寄存器返回。但在 LiteOS 中, 系统在任务初始化时将任务的上下文初始化情况如下: R0 寄存器被设置为任务的 ID, PC 寄存器被设置为 `osTaskEntry()`, LR 寄存器被设置为 `osTaskExit()`。`osTaskEntry()` 函数中会调用用户的任务函数, 并在返回后调用 `LOS_TaskDelete()` 删除自己, 所以尽管 LR 寄存器被设置为 `osTaskExit()`, 但并不会真正返回到这个函数中, 这就大大提高了代码的健壮性。当然这些操作对用户来说是不可见的, 读者可以将 `osTaskEntry` 函数理解为是哨兵, 在用户函数退出的时候, 哨兵发现了, 就把自己删除掉而不是通过 LR 返回到 `osTaskExit` 中。

代码清单 3.5 (5): 任务栈大小, 单位为字节。使用动态内存创建任务时, 只需要知道任务栈的大小即可, 因为它的任务栈空间是在任务创建时由系统动态分配的。如果系统的内存不足以分配足够大的任务栈, 那么该任务将无法被创建, 同时返回错误代码, 用户就可以根据错误代码调整系统的内存。

代码清单 3.5 (6): 使用 `LOS_TaskCreate()`函数创建一个任务, 需要传递用户定义的任务 ID 变量 `Test1_Task_Handle` 的地址, 在创建任务完成后, 系统将返回一个任务 ID, 任务参数结构体 `task_init_param` 配置的参数作为创建任务所需的参数。

代码清单 3.5 (7): 返回任务创建的结果, 如果是 `LOS_OK`, 则表示任务创建成功, 否则表示任务创建失败, 并且返回错误代码。

3.3 单任务例程 `main.c` 文件主要内容

把任务主体、任务 ID 变量、任务控制块这三部分的代码统一放到 `main.c` 中实现, 就可以组成一个系统可以运行的任务, 并且使用串口打印调试信息以便观察, 如代码清单 3.6 所示。

代码清单 3.6 `main.c` 文件内容全貌

```
1 /* LiteOS 头文件 */
2 #include "los_sys.h"
3 #include "los_task.ph"
4 /* 板级外设头文件 */
5 #include "usart.h"
6 #include "led.h"
7
8 /* 定义任务 ID 变量 */
9 UINT32 Test1_Task_Handle;
10
11 /* 函数声明 */
12 static UINT32 AppTaskCreate(void);
13 static UINT32 Creat_Test1_Task(void);
14
15 static void Test1_Task(void);
16 static void BSP_Init(void);
17
18 /*****
19 *主函数
20 *功能: (1) 开发板硬件初始化
21 *       (2) 创建 App 应用任务
22 *       (3) 启动 LiteOS, 开始多任务调度, 启动失败则输出错误信息
23 *****/
24 int main(void)
25 {
26     UINT32 uwRet = LOS_OK; //定义一个任务创建的返回值, 默认为创建成功
27
28     /* 板载相关初始化 */
29     BSP_Init();
30
31     printf("这是一个 LiteOS 动态创建单任务例程! \r\n");
32
33     /* LiteOS 内核初始化 */
34     uwRet = LOS_KernelInit();
35 }
```

```

36     if(uwRet != LOS_OK) {
37         printf("LiteOS 核心初始化失败! 失败代码 0x%X\n", uwRet);
38         return LOS_NOK;
39     }
40
41     uwRet = AppTaskCreate();
42     if(uwRet != LOS_OK) {
43         printf("AppTaskCreate创建任务失败! 失败代码 0x%X\n", uwRet);
44         return LOS_NOK;
45     }
46
47     /* 开启 LiteOS任务调度 */
48     LOS_Start();
49
50     //正常情况下不会执行到这里
51     while(1);
52
53 }
54
55 /*****
56 *函数:  AppTaskCreate
57 *功能:  任务创建, 为了方便管理, 所有的任务创建函数都可以放在这个函数里面
58 *****/
59 static UINT32 AppTaskCreate(void)
60 {
61     /* 定义一个返回类型变量, 初始化为 LOS_OK */
62     UINT32 uwRet = LOS_OK;
63
64     uwRet = Creat_Test1_Task();
65     if(uwRet != LOS_OK) {
66         printf("Test1_Task任务创建失败! 失败代码 0x%X\n", uwRet);
67         return uwRet;
68     }
69     return LOS_OK;
70 }
71
72 /*****
73 *函数:  Creat_Test1_Task
74 *功能:  创建Test1_Task任务
75 *****/
76 static UINT32 Creat_Test1_Task()
77 {
78     //定义一个创建任务的返回类型, 初始化为创建成功的返回值
79     UINT32 uwRet = LOS_OK;
80
81     //定义一个用于创建任务的参数结构体
82     TSK_INIT_PARAM_S task_init_param;
83
84     task_init_param.usTaskPrio = 5; /* 任务优先级, 数值越小, 优先级越高 */
85     task_init_param.pcName = "Test1_Task"; /* 任务名 */
86     task_init_param.pfnTaskEntry = (TSK_ENTRY_FUNC)Test1_Task;
87     task_init_param.uwStackSize = 1024; /* 任务栈大小 */
88
89     uwRet = LOS_TaskCreate(&Test1_Task_Handle, &task_init_param);
90     return uwRet;
91 }
92
93 /*****
94 *函数:  Test1_Task
95 *功能:  Test1_Task任务实现
96 *****/
97 static void Test1_Task(void)
98 {
99     /* 任务都是一个无限循环, 不能返回 */
100     while(1) {
101         LED0 = !LED0;
102         printf("任务1运行中, 每1000ticks打印一次信息\r\n");
103         LOS_TaskDelay(1000);

```

```

104     }
105 }
106
107 /*****
108 *函数名:   BSP_Init
109 *功能:     板级外设初始化, 所有开发板上的初始化均可放在这个函数里面
110 *****/
111 static void BSP_Init(void)
112 {
113     /* STM32 中断优先级分组为 4, 即 4bit都用来表示抢占优先级, 取值为: 0~15
114     * 优先级分组只需要分组一次即可, 以后如果有其他的任务需要用到中断,
115     * 都统一用这个优先级分组, 千万不要再分组, 切忌。
116     */
117     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
118
119     /* 初始化与 LED 连接的硬件接口 */
120     LED_Init();
121
122     /* 串口初始化 */
123     uart_init(72,115200);
124 }
125 /*****END OF FILE*****/

```

3.4 创建多任务

在上一节中介绍了创建单个任务的过程, 那么本节将介绍创建多个任务的过程, 创建多个任务与创建单个任务的过程其实是相同的流程——定义任务 ID 变量、实现函数主体、配置任务相关信息、创建任务。本节将创建两个任务, 两个任务的优先级不同, 任务 1 实现 LED1 灯闪烁, 任务 2 实现 LED2 闪烁, 两个 LED 闪烁的频率不一样, 并且在串口中输出相应的调试信息, 具体实现如代码清单 3.7 加粗部分所示。

代码清单 3.7 创建多任务

```

1  /* LiteOS 头文件 */
2  #include "los_sys.h"
3  #include "los_task.ph"
4  /* 板级外设头文件 */
5  #include "sys.h"
6  #include "usart.h"
7  #include "led.h"
8
9  /*定义任务 ID变量 */
10 UINT32 Test1_Task_Handle;
11 UINT32 Test2_Task_Handle;
12
13 /* 函数声明 */
14 static UINT32 AppTaskCreate(void);
15 static UINT32 Creat_Test1_Task(void);
16 static UINT32 Creat_Test2_Task(void);
17
18 static void Test1_Task(void);
19 static void Test2_Task(void);
20 static void BSP_Init(void);
21
22

```

```

23 /*****
24 *主函数
20 *功能:   (1) 开发板硬件初始化
21 *         (2) 创建 App 应用任务
22 *         (3) 启动 LiteOS, 开始多任务调度, 启动失败则输出错误信息
23 *****/
24 int main(void)
25 {
26     UINT32 uwRet = LOS_OK;    //定义一个任务创建的返回值, 默认为创建成功
27
28     /* 板载相关初始化 */
29     Stm32_Clock_Init(9);    //系统时钟设置
30     BSP_Init();
31
32     printf("这是一个 LiteOS 创建多任务例程!  \r\n");
33
34     /* LiteOS 内核初始化 */
35     uwRet = LOS_KernelInit();
36
37     if (uwRet != LOS_OK) {
38         printf("LiteOS 核心初始化失败! 失败代码 0x%X\n", uwRet);
39         return LOS_NOK;
40     }
41
42     uwRet = AppTaskCreate();
43     if (uwRet != LOS_OK) {
44         printf("AppTaskCreate创建任务失败! 失败代码 0x%X\n", uwRet);
45         return LOS_NOK;
46     }
47
48     /* 开启 LiteOS任务调度 */
49     LOS_Start();
50
51     //正常情况下不会执行到这里
52     while (1);
53 }
54
55 /*****
56 *函数名:   AppTaskCreate
57 *功能:     任务创建, 为了方便管理, 所有的任务创建函数都可以放在这个函数里面
58 *****/
59 static UINT32 AppTaskCreate(void)
60 {
61     /* 定义一个返回类型变量, 初始化为 LOS_OK */
62     UINT32 uwRet = LOS_OK;
63
64     uwRet = Creat_Test1_Task();
65     if(uwRet != LOS_OK) {
66         printf("Test1_Task任务创建失败! 失败代码 0x%X\n", uwRet);
67         return uwRet;
68     }
69
70     uwRet = Creat_Test2_Task();
71     if (uwRet != LOS_OK) {
72         printf("Test2_Task任务创建失败! 失败代码 0x%X\n", uwRet);
73         return uwRet;
74     }
75     return LOS_OK;
76 }
77
78 /*****
79 *函数名:   Creat_Test1_Task
80 *功能:     创建Test1_Task任务
81 *****/
82 static UINT32 Creat_Test1_Task()
83 {
84     //定义一个创建任务的返回类型, 初始化为创建成功的返回值
85     UINT32 uwRet = LOS_OK;
86

```

```

87 //定义一个用于创建任务的参数结构体
88 TSK_INIT_PARAM_S task_init_param;
89
90 task_init_param.usTaskPrio = 5; /* 任务优先级, 数值越小, 优先级越高 */
91 task_init_param.pcName = "Test1_Task"; /* 任务名 */
92 task_init_param.pfnTaskEntry = (TSK_ENTRY_FUNC)Test1_Task;
93 task_init_param.uwStackSize = 1024; /* 栈大小 */
94
95 uwRet = LOS_TaskCreate(&Test1_Task_Handle, &task_init_param);
96 return uwRet;
97 }
98 /*****
99 *函数名: Creat_Test2_Task
100 *功能: 创建Test2_Task任务
101 *****/
102 static UINT32 Creat_Test2_Task()
103 {
104     // 定义一个创建任务的返回类型, 初始化为创建成功的返回值
105     UINT32 uwRet = LOS_OK;
106     TSK_INIT_PARAM_S task_init_param;
107
108     task_init_param.usTaskPrio = 4; /* 任务优先级, 数值越小, 优先级越高 */
109     task_init_param.pcName = "Test2_Task"; /* 任务名*/
110     task_init_param.pfnTaskEntry = (TSK_ENTRY_FUNC)Test2_Task;
111     task_init_param.uwStackSize = 1024; /* 栈大小 */
112
113     uwRet = LOS_TaskCreate(&Test2_Task_Handle, &task_init_param);
114
115     return uwRet;
116 }
117
118 /*****
119 *函数名: Test1_Task
120 *功能: Test1_Task任务实现
121 *****/
122 static void Test1_Task(void)
123 {
124     /* 任务都是一个无限循环, 不能返回 */
125     while(1) {
126         LED0 = !LED0;
127         printf("任务1运行中, 每1000ticks打印一次信息\r\n");
128         LOS_TaskDelay(1000);
129     }
130 }
131 /*****
132 *函数名: Test2_Task
133 *功能: Test2_Task任务实现
134 *****/
135 static void Test2_Task(void)
136 {
137     /* 任务都是一个无限循环, 不能返回 */
138     while(1) {
139         LED1 = !LED1;
140         printf("任务2运行中, 每500ticks打印一次信息\n");
141         LOS_TaskDelay(500);
142     }
143 }
144
145 /*****
146 *函数名: BSP_Init
147 *功能: 板级外设初始化, 所有开发板上的初始化均可放在这个函数里面
148 *****/
149 static void BSP_Init(void)
150 {
151     /* STM32中断优先级分组为4, 即4bit都用来表示抢占优先级, 取指为0~15
152     * 优先级分组只需要分组一次即可, 以后如果有其他的任务需要用到中断,
153     * 都统一用这个优先级分组, 千万不要再分组, 切忌。*/
154     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_4);

```

```

156
157  /* LED 初始化 */
157    LED_Init();
158
159  /* 串口初始化 */
160    uart_init(72,115200);
161 }

```

本小节的例程只创建了两个任务，如果读者想要创建 3 个、4 个甚至更多的任务，其过程也是一样的。容易忽略的地方是任务栈的大小、优先级等配置，读者可以根据创建任务的重要性与复杂度等配置不同的任务栈空间与任务优先级。

3.5 LiteOS 的启动流程

3.5.1 启动文件

在系统上电的时候，系统第一个执行的是启动文件中由汇编编写的复位函数 `Reset_Handler`，如代码清单 3.8 所示。复位函数会进行系统时钟的初始化、接着调用运行时库函数 `_main` 初始化系统的运行的环境，`_main` 是 keil 提供的类似 gcc 的 `init_array` 的初始化分散加载相关内容的函数，这个函数并非是 C 库函数，可以称之为运行时库函数。

启动文件由汇编编写，是系统上电复位后第一个执行的程序，主要做了以下工作：

- (1) 堆栈和堆栈指针初始值（`initial_sp`）设置；
- (2) 初始化中断向量表；
- (4) 配置系统时钟（不同版本存在区别）；
- (5) 调用 `__main` 初始化用户栈，从而最终调用 `main` 函数去到用户编写的 C

代码

代码清单 3.8 `Reset_Handler` 函数

```

1 Reset_Handler  PROC
2                 EXPORT Reset_Handler             [WEAK]
3                 IMPORT  main
4 ;寄存器版本代码，因为没有用到 SystemInit 函数，所以注释掉以下代码为防止报错！
5 ;库函数版本代码，建议加上这里（外部必须实现 SystemInit 函数），以初始化 stm32 时钟等。
6                 ;IMPORT SystemInit
7                 ;LDR     R0, =SystemInit
8                 ;BLX     R0
9                 LDR     R0, =__main
10                BX      R0
11                ENDP

```

3.5.2 LiteOS 初始化

在 main 函数中，用户需要完成硬件初始化，然后调用 LOS_KernelInit() 函数对 LiteOS 核心部分（内核）进行初始化，因为只有在 LiteOS 内核初始化完成之后，用户才能调用系统相关的函数进行创建任务、消息队列、信号量、互斥锁、事件等操作。如果 LiteOS 内核初始化失败，则返回错误代码。LOS_KernelInit() 函数完成的部分操作如下：

（1）根据 target_config.h 中的 LOSCFG_BASE_CORE_TSK_LIMIT 宏定义配置系统最大支持的任务个数，默认为 LOSCFG_BASE_CORE_TSK_LIMIT+1，包括空闲任务 IDLE。

（2）初始化 LiteOS 内存管理模块，系统管理的内存大小为 OS_SYS_MEM_SIZE，可以由用户进行设置。

（3）如果在 target_config.h 中使能 LOSCFG_PLATFORM_HWI 宏定义，则进行硬件中断模块的初始化。表示 LiteOS 接管系统中断，使用中断时首先需要注册中断，否则将无法响应中断；反之 LiteOS 没有接管中断，中断的响应同裸机系统。

（4）初始化任务模块相关的内容，如分配内存、初始化链表，为后续创建任务做准备工作。

（5）根据 target_config.h 中 LOSCFG_BASE_IPC_SEM 宏定义，决定是否需要初始化信号量相关内容，如分配内存、初始化信号量链表等。

（6）根据 target_config.h 中 LOSCFG_BASE_IPC_MUX 宏定义，决定是否需要互斥锁进行相关初始化，如分配内存、初始化链表等。

（7）根据 target_config.h 中 LOSCFG_BASE_IPC_QUEUE 宏定义，决定是否需要消息队列进行相关初始化，如分配内存、初始化链表等。

（8）创建一个空闲任务以保证系统的正常运行。

注意：系统支持最大的信号量、互斥锁、消息队列个数由用户决定，并且在内核初始化的时候会分配对应控制块所需的内存空间，用户可以根据系统的需求进行裁剪，以减少系统的内存开销。

3.5.3 创建任务与开启任务调度

在系统完成一系列相关的初始化之后，LiteOS 就可以正常启动了，读者可以

使用系统提供的 API 来创建任务、内核对象等，然后开启任务调度，并且用户可以在任务中使用内核资源完成需要的功能，伪代码如代码清单 3.9 加粗部分所示。

代码清单 3.9 创建任务与开启调度（伪代码）

```
1  /* LiteOS 核心初始化 */
2  uwRet = LOS_KernelInit();
3  if (uwRet != LOS_OK)
4  {
5      printf("LiteOS 核心初始化失败!  \n");
6      return LOS_NOK;
7  }
8  /* 创建Test1_Task任务创建失败 */
9  uwRet = Creat_Test1_Task();
10 if(uwRet != LOS_OK)
11 {
12     printf("Test1_Task任务创建失败!  \n");
13     return LOS_NOK;
14 }
15 /* 创建Test2_Task任务创建失败 */
16 uwRet = Creat_Test2_Task();
17 if(uwRet != LOS_OK)
18 {
19     printf("Test2_Task任务创建失败!  \n");
20     return LOS_NOK;
21 }
22 /* 开启LiteOS任务调度 */
23 LOS_Start();
24
25 /* Test1_Task 任务入口函数 */
26 void Test1_Task( void *arg )
27 {
28     while(1)
29     {
30         /* 任务主体，必须有阻塞的情况出现 */
31     }
32 }
33
34 /* Test2_Task 任务入口函数 */
35 void Test2_Task( void *arg )
36 {
37     while(1)
38     {
39         /* 任务主体，必须有阻塞的情况出现 */
40     }
41 }
```

任务创建完成时默认是就绪态（OS_TASK_STATUS_READY），只有处于就绪态的任务才能参与系统的调度。系统调度器的开启由 LOS_Start()函数来实现。系统在开启调度器后，任务就会进行切换，任务的切换包括从已经创建完成的就绪任务列表中选择一个优先级最高的任务开始运行、切出任务上文保存、切入任务下文恢复等动作。

LOS_Start()函数打开系统必要的时钟以保证系统能获得正常的时基，系统时钟节拍根据宏定义 OS_SYS_CLOCK 与 LOSCFG_BASE_CORE_TICK_PER

_SECOND 进行设置。并配置 SysTick 与 PendSV 的优先级均为“0xF0”，即最低的优先级，避免 SysTick 与 PendSV 系统异常抢占系统中的其他重要中断。

任务主体通常是一个不带返回值并且是无限循环的 C 函数，函数体中必须有阻塞（比如延时）的情况出现，否则该任务会一直在 while 循环中占有 CPU，导致系统中优先级比它低的任务无法获得 CPU 的使用权。也就意味着如果高优先级任务中没有阻塞情况，除非任务结束，否则低优先级任务无法执行。

第 4 章 任务管理

任务管理是 LiteOS 的核心组成部分，本章主要介绍任务及调度器相关的概念，分析任务相关函数的实现过程。

4.1 基本概念

4.1.1 任务的基本概念

从系统的角度看，任务是竞争系统资源的最小运行单元。LiteOS 是一个支持多任务的操作系统。在 LiteOS 中，任务可以使用 CPU、内存空间等系统资源，并独立于其他任务运行。理论上任何数量的任务都可以共享同一个优先级，处于就绪态的多个最高优先级任务将会以时间片切换的方式共享 CPU。

简而言之：LiteOS 的任务可认为是一系列独立任务的集合，每个任务在独立的环境中运行。在任何时刻，有且只有一个任务处于运行态，由 LiteOS 调度器决定哪个任务可以运行。调度器的主要职责是找到处于就绪态的最高优先级任务，并且在任务切入切出时保存任务上下文内容（寄存器值、任务栈数据），为了实现这点，LiteOS 的每个任务都需要有独立的任务栈，当任务切出时，它的上下文环境会被保存在任务栈中，当任务恢复运行时，就能从任务栈中正确的恢复上次的运行环境。系统的任务越多，则需要的 SRAM 就越大，一个系统能够运行多少个任务，取决于系统的 SRAM 大小。

任务享有独立的栈空间，系统决定任务的状态，它表示任务是否可以运行，同时还能使用内核的 IPC 通信资源，实现中断与任务、任务与任务之间的通信。

LiteOS 支持抢占式调度机制，高优先级的任务可打断低优先级任务，低优先级任务必须在高优先级任务阻塞或结束后才能得到调度，同时 LiteOS 也支持时间片轮转调度机制。

任务通常会运行在一个死循环中不会退出，如果一个任务不再需要运行时，可以调用 LiteOS 中的任务删除函数将其删除。

LiteOS 的任务默认有 32 个优先级(0-31)，最高优先级为 0，最低优先级为 31。

4.1.2 任务调度器的基本概念

LiteOS 提供的任务调度器是基于优先级的全抢占式调度，在系统中除了中断处理函数、调度器上锁和处于临界段中的情况是不可抢占之外，系统的其他部分都是可以抢占的。系统默认可以支持 32 个优先级（0~31），优先级数值越大的任务优先级越低，31 为最低优先级，分配给空闲任务使用，一般不建议用户来使用这个优先级。在一些资源比较紧张的系统中，可以根据实际情况选择系统支持的任务优先级个数，创建合适的任务，以节约内存资源。在系统中，当有比当前任务优先级更高的任务就绪时，当前任务将立刻被切出，高优先级任务抢占获取 CPU 的使用权。

LiteOS 内核中也允许创建相同优先级的任务，相同优先级的任务采用时间片轮转方式进行调度（也就是通常说的分时调度器），时间片轮转调度仅在当前系统有多个最高优先级就绪任务存在的情况下才有效。为了保证系统的实时性，系统尽最大可能地保证高优先级的任务得以运行，任务调度的原则是一旦任务状态发生了改变，并且当前运行的任务优先级小于就绪列表中最高优先级任务时，立刻进行任务切换（除非当前系统处于中断处理程序中或禁止任务切换的状态）。

4.1.3 任务状态的概念

LiteOS 系统中的任务都有多种运行状态。系统初始化完成后，创建的任务就可以在系统中竞争资源，由内核进行调度。

任务状态通常分为以下四种。

（1）就绪态（Ready）：该任务处于就绪列表中，就绪的任务已经具备执行的能力，只等待调度器进行调度，新创建的任务会被初始化为该状态。

（2）运行态（Running）：该状态表明任务正在执行，此时它占用 CPU，LiteOS 调度器选择运行的永远是处于最高优先级的就绪态任务，当任务被运行的一刻，它的任务状态就变成了运行态。

（3）阻塞态（Blocked）：如果任务当前正在等待某个时序或外部中断，那么该任务处于阻塞状态，它不处于就绪列表中，无法得到调度器的调度。阻塞态包含任务被挂起（如调用 `LOS_TaskSuspend()` 函数）、任务被延时（如调用 `LOS_TaskDelay()` 函数）、任务正在等待信号量（SEM）、读写队列（QUEUE）

或者等待读写事件（EVENT）等。

（4）**退出态**（Dead）：该任务运行结束，等待系统回收资源。

4.1.4 任务状态迁移

LiteOS 系统中的每一个任务都有多种运行状态，它们之间的转换关系是怎么样的呢？从运行态任务变成阻塞态，或者从阻塞态变成就绪态，这些任务状态是怎么迁移的呢，下面就来了解任务状态的迁移吧，如图 4.1 所示。

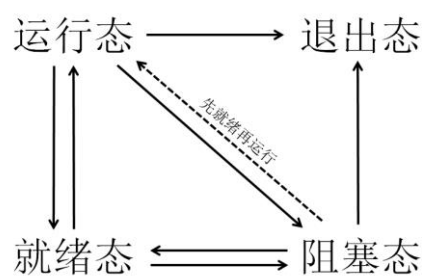


图 4.1 任务状态示意图

创建任务→就绪态：任务创建完成后进入就绪态，表明任务已准备就绪，随时可以运行，只等待调度器进行调度。

就绪态→运行态：任务创建后进入就绪态，发生任务切换时，就绪列表中最高优先级的任务被执行，从而进入运行态，但此刻该任务依旧在就绪列表中。

运行态→就绪态：**有更高优先级任务创建或者恢复后**，会发生任务调度，**此刻就绪列表中最高优先级任务变为运行态，而原先运行的任务由运行态变为就绪态**，仍处于就绪列表中，**等待最高优先级的任务运行完毕或阻塞后继续运行**（CPU 使用权被更高优先级的任务抢占了）。

运行态→阻塞态：正在运行的任务发生阻塞时（挂起、延时、读信号量等），**该任务会从就绪列表中删除**。任务状态由运行态变成阻塞态，然后发生任务切换，系统运行就绪列表中最高优先级任务。

阻塞态→就绪态（阻塞态→运行态）：阻塞的任务被恢复后（任务恢复、延时时间超时、读信号量超时或读到信号量等），此时被恢复的任务会被加入就绪列表，从而由阻塞态变成就绪态；如果此时被恢复任务的优先级高于正在运行任务的优先级，则会发生任务切换，将该任务再次转换任务状态，由就绪态变成运行态。

就绪态→阻塞态：任务也有可能在就绪态时被阻塞（挂起），此时任务状态会由就绪态变为阻塞态，该任务从就绪列表中删除，不会参与系统调度，直到该任务被恢复就绪态。

运行态、阻塞态→退出态：调用系统中删除任务的函数，无论是处于何种状态的任务都将变为退出态。

4.2 常用的任务函数讲解

4.2.1 任务创建函数 `LOS_TaskCreate()`

在前面的章节中，本资料已经讲解了任务创建函数 `LOS_TaskCreate()` 的使用，`LOS_TaskCreate()` 函数的主要功能如下：

- （1）定义一个新创建任务的任务控制块结构体指针，用于保存新创建任务的任务信息。
- （2）创建新任务并返回一个任务 ID，通过任务 ID 获取对应任务控制块的信息，同时回收之前已删除任务的任务控制块和任务栈。
- （3）将新创建的任务设置为就绪态。
- （4）将新创建任务按照设定的优先级顺序插入任务就绪列表。
- （5）如果开启了任务调度，并且调度器没有被上锁，则判断新建的任务优先级是否比当前任务的优先级更高，是则进行一次任务调度，否则将返回任务创建成功标志。

在 Cortex-M 系列处理器中，LiteOS 的调度是利用 PendSV 进行任务调度的，LiteOS 向 0xE000ED04 这个地址写入 0x10000000，即将 SCB（系统控制块，基地址 0xE000ED00）中的 ICSR（中断控制状态寄存器，偏移地址 0x04）的第 28 位置 1，触发 PendSV 中断，真正的任务切换是在 PendSV 服务程序中进行的，如图 4.2 所示。

4.4.14 SCB register map

The table provides shows the System control block register map and reset values. The base address of the SCB register block is **0xE000 ED00**.

Table 47. SCB register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	SCB_CPUID	Implementer																Constant				PartNo				Revision							
	Reset Value	0	1	0	0	0	0	0	1	0	0	0	1	1	1	1	1	1	1	0	0	0	0	1	0	0	0	1	1	0	0	0	1
0x04	SCB_ICSR	NMIPENDSET	Reser ved		PENDSVSET	PENDSVCLR	PENDSTSET	PENDSTCLR	Reser ved	ISRPENDING	VECTPENDING[9:0]										RETOBASE	Reser ved		VECTACTIVE[8:0]									
	Reset Value				0	0	0	0			0	0	0	0	0	0	0	0	0	0				0	0	0	0	0	0	0	0	0	0

图 4.2 任务调度通过 ICSR 将 PendSV 置 1

4.2.2 任务删除函数 LOS_TaskDelete()

在 LiteOS 中支持显式删除任务，当任务不需要的时候，可以删除它。删除任务后，LiteOS 会回收任务的相关资源。LOS_TaskDelete()函数的主要功能如下：

- (1) 如果要删除的任务的任务状态是 OS_TASK_STATUS_UNUSED ，表示任务尚未创建，系统无法删除，将返回错误代码。
- (2) 如果要删除的任务正在运行且调度器已经被上锁，系统会将任务解锁，然后进行删除操作。
- (3) 如果要删除的任务在就绪态，那么将要删除的任务从就绪列表中移除，并且取消任务的就绪状态。
- (4) 如果要删除的任务在阻塞态，那么将要删除的任务从阻塞列表中删除。
- (5) 如果要删除的任务正在处于延时状态或者任务正在等待信号量、事件等阻塞超时状态，那么从延时列表中删除任务。
- (6) 系统重新在就绪列表中寻找处于就绪态的最高优先级任务，保证系统能正常运行。
- (7) 如果删除的任务是当前正在运行的任务，因为删除任务以后要调度新的任务运行，而调度的过程需要当前任务的参与，所以还不能直接将当前任务彻底删除掉，只是将任务添加到系统的回收列表中，在创建任务的时候再将回收列表中的任务进行回收，而当前任务需要继续执行，直到系统调度完成；如果被删除的任务不是当前任务，那么直接将任务状态变为未使用状态。
- (8) 归还任务控制块，将任务控制块插入系统可用任务链表中，便于以后能再创建任务。

(9) 将任务控制块的内存进行释放，将任务的栈顶指针指向 NULL。

(10) 如果删除成功则返回 LOS_OK，否则将返回错误代码。

4.2.3 任务延时函数 LOS_TaskDelay()

延时函数是在使用操作系统的时候经常用到的函数，延时函数的作用是将调用延时函数的任务进入阻塞态而放弃 CPU 的使用权，这样系统中其他优先级较低的任务就能获得 CPU 的使用权。否则，高优先级任务一直占用 CPU，导致系统无法实现任务切换，比它优先级低的任务将永远得不到执行，延时的基本单位为时钟节拍 Tick，配置 LOSCFG_BASE_CORE_TICK_PER_SECOND 宏定义可改变 Tick，如果 LOSCFG_BASE_CORE_TICK_PER_SECOND 配置为 1000，那么一秒钟有 1000 个 Tick，一个 Tick 为 1ms，延时函数的主要功能如下：

(1) 判断是否在中断中延时，如果在**中断中进行延时**，这将是**非法的**，延时函数会返回错误代码，因为 LiteOS 不允许在中断中调用延时操作。

(2) 如果在**调度器被锁定时进行延时**，这也是**非法的**，因为延时操作需要依赖调度器的调度，因此延时函数也会返回错误代码。

(3) 如果要进行**0 个 Tick 的延时**，那么当前任务将主动放弃 CPU 的使用权，**进行一次强制切换任务**。

(4) 如果任务可以进行延时，LiteOS 将调用延时函数的任务从就绪列表中删除，然后将该任务添加到延时链表中，最后将任务的状态变为延时状态（阻塞态），当延时的时间到达，任务将从阻塞态直接变为就绪态。最后，LiteOS 进行一次任务的切换，再返回 LOS_OK 表示延时成功。

注意：在每个任务的循环中必须要有阻塞的出现，否则，比该任务优先级低的任务是永远无法获得 CPU 的使用权的。

4.2.4 任务挂起函数 LOS_TaskSuspend()

LiteOS 支持挂起指定任务，被挂起的任务不会得到 CPU 使用权，不管该任务具有什么优先级。

调用 LOS_TaskSuspend() 函数挂起任务的次数是不会累计的，即使多次调用 LOS_TaskSuspend() 函数将一个任务挂起，也只需调用一次任务恢复函数

LOS_TaskResume()就能使挂起的任务解除挂起状态。任务挂起是经常使用的一个函数，如果用户想要某个任务长时间暂停运行，就可以使用挂起函数将该任务挂起，任务挂起函数的主要功能如下：

（1）通过任务 ID 获取对应的任务控制块，判断要挂起任务的状态，如果是未使用状态，或者该任务已经被挂起了，则返回错误代码。如果任务在运行中并且调度器已经被上锁了，那么将无法进行挂起任务，也会返回错误代码。

（3）如果任务处于就绪态，则可以进行挂起任务，挂起函数将任务从就绪列表中删除，并解除就绪态，将任务的状态变为挂起态。

（4）进行一次任务调度。

4.2.5 任务恢复函数 LOS_TaskResume()

任务恢复就是让挂起的任务重新进入就绪状态，恢复的任务会保留挂起前的状态信息，在恢复的时候继续运行。如果被恢复任务在所有就绪态任务中，处于系统中的最高优先级，那么系统将进行一次任务切换。任务恢复函数 LOS_TaskResume()的主要功能如下所示：

（1）根据任务 ID 获取任务控制块，通过任务控制块判断要恢复任务的状态，如果是未使用状态，或者是未挂起状态，则返回错误代码。

（2）将任务的状态从阻塞态解除，转换为就绪态，并根据任务的优先级数值添加到就绪列表中。

（3）如果调度器已经在运行了，则发起一次任务调度，在任务调度中会寻找处于就绪态的最高优先级任务，**如果被恢复的任务刚好是就绪态任务中的最高优先级，那么系统会立即执行该任务。**

4.3 常用 Task 错误代码说明

在 LiteOS 中，与任务相关的函数大多数都会有返回值，其返回值是一些错误代码，方便用户进行调试，本资料列出一些常见的错误代码与参考解决方案，如表 4.1 所示。

表 4.1 常用 Task 函数返回的错误代码说明

序号	定义	描述	参考解决方案
1	LOS_ERRNO_TSK_NO_MEMORY (0x03000200)	内存空间不足	分配更大的内存
2	LOS_ERRNO_TSK_PTR_NULL (0x02000201)	任务参数为空	检查任务参数
3	LOS_ERRNO_TSK_STKSZ_NOT_ALIGN (0x02000202)	任务栈未对齐	对齐任务栈
4	LOS_ERRNO_TSK_PRIOR_ERROR (0x02000203)	不正确的任务 优先级	检查任务优先级
5	LOS_ERRNO_TSK_ENTRY_NULL (0x02000204)	任务入口函数 为空	定义任务入口函数
6	LOS_ERRNO_TSK_NAME_EMPTY (0x02000205)	任务名为空	设置任务名
7	LOS_ERRNO_TSK_STKSZ_TOO_SMALL (0x02000206)	任务栈太小	扩大任务栈
8	LOS_ERRNO_TSK_ID_INVALID (0x02000207)	无效的任务ID	检查任务ID
9	LOS_ERRNO_TSK_ALREADY_SUSPENDED (0x02000208)	任务已经被挂 起	等待这个任务被恢 复后，再去尝试挂 起这个任务
10	LOS_ERRNO_TSK_NOT_SUSPENDED (0x02000209)	任务未被挂起	挂起这个任务
11	LOS_ERRNO_TSK_NOT_CREATED (0x0200020a)	任务未被创建	创建这个任务
12	LOS_ERRNO_TSK_DELETE_LOCKED (0x0300020b)	删除任务时,任 务处于被锁状 态	等待解锁任务之后 再进行删除操作
13	LOS_ERRNO_TSK_MSG_NONZERO (0x0200020c)	任务信息非零	暂不使用该错误代 码
14	LOS_ERRNO_TSK_DELAY_IN_INT (0x0300020d)	中断期间，进 行任务延时	等待退出中断后再 进行延时操作
15	LOS_ERRNO_TSK_DELAY_IN_LOCK (0x0200020e)	任务被锁的状 态下，进行延时	等待解锁任务之后再 进行延时操作
16	LOS_ERRNO_TSK_YIELD_INVALID_TASK (0x0200020f)	将被排入行程 的任务是无效 的	检查这个任务
17	LOS_ERRNO_TSK_YIELD_NOT_ENOUGH_TASK (0x02000210)	没有或者仅有一个可用任务 能进行行程安 排	增加任务数
18	LOS_ERRNO_TSK_TCB_UNAVAILABLE (0x02000211)	没有空闲的任 务控制块可用	增加任务控制块数量
19	LOS_ERRNO_TSK_HOOK_NOT_MATCH (0x02000212)	任务的钩子函 数不匹配	暂不使用该错误代码
20	LOS_ERRNO_TSK_HOOK_IS_FULL (0x02000213)	任务的钩子函 数数量超过界 限	暂不使用该错误代码
21	LOS_ERRNO_TSK_OPERATE_IDLE (0x02000214)	这是个 IDLE 任 务	检查任务 ID, 不要试图 操作 IDLE 任务

22	LOS_ERRNO_TSK_SUSPEND_LOCKED (0x03000215)	将被挂起的任务处于被锁状态	等待任务解锁后再尝试挂起任务
23	LOS_ERRNO_TSK_FREE_STACK_FAILED (0x02000217)	任务栈 free 失败	该错误代码暂不使用
24	LOS_ERRNO_TSK_STKAREA_TOO_SMALL (0x02000218)	任务栈区域太小	该错误代码暂不使用
25	LOS_ERRNO_TSK_ACTIVE_FAILED (0x03000219)	任务触发失败	创建一个 IDLE 任务后执行任务转换
26	LOS_ERRNO_TSK_CONFIG_TOO_MANY (0x0200021a)	过多的任务配置项	该错误代码暂不使用
27	LOS_ERRNO_TSK_STKSZ_TOO_LARGE (0x02000220)	任务栈大小设置过大	减小任务栈大小
28	LOS_ERRNO_TSK_SUSPEND_SWTMR_NOT_ALLOWED (0x02000221)	不允许挂起软件定时器任务	检查任务 ID，不要试图挂起软件定时器任务

4.4 常用任务函数的使用方法

4.4.1 任务创建函数 LOS_TaskCreate()

LOS_TaskCreate()函数原型如代码清单 4.1 所示。创建任务函数是创建每个独立任务的时候是必须使用的，在使用函数的时候，需要提前定义任务 ID 变量，并且要配置实现任务创建的初始化参数数据结构体(TSK_INIT_PARAM_S)变量，如代码清单 4.2 加粗部分所示。如果任务创建成功，则返回 LOS_OK，否则返回对应的错误代码。

代码清单 4.1 LOS_TaskCreate()函数原型

```
1 UINT32 LOS_TaskCreate(UINT32 *puwTaskID, TSK_INIT_PARAM_S *pstInitParam);
```

代码清单 4.2 自定义实现任务的相关配置

```
1 UINT32 Test1_Task_Handle; /* 定义任务 ID 变量 */
2 TSK_INIT_PARAM_S task_init_param; /* 自定义任务配置的相关参数 */
3
4 task_init_param.usTaskPrio = 5; /* 优先级，数值越小，优先级越高 */
5 task_init_param.pcName = "Test1_Task"; /* 任务名，字符串形式，方便调试 */
6 task_init_param.pfnTaskEntry = (TSK_ENTRY_FUNC)Test1_Task; /* 任务函数名 */
7 task_init_param.uwStackSize = 0x1000; /* 栈大小，单位为字，即 4 个字节 */
8
9 uwRet = LOS_TaskCreate(&Test1_Task_Handle, &task_init_param); /* 创建任务 */
```

任务初始化参数数据结构体 TSK_INIT_PARAM_S 在 los_task.h 中定义，其内部的配置参数具体作用如代码清单 4.3 所示，读者可以根据自己的任务需要

来配置，重要的任务优先级可以设置高一点，任务栈可以设置大一点，防止溢出导致系统崩溃，若指定的任务栈大小为 0，则系统使用 target_config.h 中的配置项 LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE 指定默认的任务栈大小，任务栈的大小按 8 字节大小对齐。

代码清单 4.3 TSK_INIT_PARAM_S 结构体

```
1 typedef struct tagTskInitParam {
2     TSK_ENTRY_FUNC pfnTaskEntry; /**< 任务的入口函数 */
3     UINT16 usTaskPrio;           /**< 任务优先级 */
4     UINT32 uwArg;                /**< 任务参数（未使用） */
5     UINT32 uwStackSize;         /**< 任务栈大小 */
6     CHAR *pcName;               /**< 任务名字 */
7     UINT32 uwResved;            /**< LiteOS 保留未使用 */
8 }TSK_INIT_PARAM_S;
```

4.4.2 任务删除函数 LOS_TaskDelete()

任务删除函数 LOS_TaskDelete()根据任务 ID 直接删除任务，任务控制块与任务栈将被系统回收，所有保存的信息都会被清空。LOS_TaskDelete()的函数原型如代码清单 4.4 所示。uwTaskID 是要删除任务的 ID。

代码清单 4.4 任务删除函数 LOS_TaskDelete()原型

```
1 LITE_OS_SEC_TEXT_INIT UINT32 LOS_TaskDelete(UINT32 uwTaskID)
```

任务删除函数的使用实例如代码清单 4.5 加粗部分所示，如果任务删除成功，则返回 LOS_OK，否则返回其他错误代码。

代码清单 4.5 任务删除函数的用法

```
1 UINT32 uwRet = LOS_OK; /* 定义一个任务的返回类型，初始化为 LOS_OK */
2
3 uwRet = LOS_TaskDelete(Test_Task_Handle)
4 if(uwRet != LOS_OK)
5 {
6     printf("任务删除失败\n");
7 }
```

4.4.3 任务延时函数 LOS_TaskDelay()

任务延时函数只有 1 个传入的参数 uwTick，它的延时单位是 Tick，支持传

入 0 个 Tick。用户根据实际情况对任务进行延时即可，其函数原型如代码清单 4.6 所示。

代码清单 4.6 延时函数任务原型

```
1 LITE_OS_SEC_TEXT UINT32 LOS_TaskDelay(UINT32 uwTick)
```

任务延时函数有几点需要注意的地方：

- (1) 延时函数不允许在中断中使用；
- (2) 延时函数不允许在任务调度被锁定的时候使用；
- (3) 如果传入 0 并且未锁定任务调度，则执行具有当前任务相同优先级的任务队列中的下一个任务，如果没有当前任务优先级的就绪任务可用，则不会发生任务调度，并继续执行当前任务；
- (4) 不允许在系统初始化之前使用该函数；
- (5) 延时函数也是有返回值的，如果使用时发生错误，可以根据返回的错误代码来进行调整；
- (6) 这种延时并不精确。

任务延时函数的使用方法如代码清单 4.7 加粗部分所示。

代码清单 4.7 延时函数的使用方法

```
1 static void Test1_Task(void)
2 {
3     /* 每个任务都是无限循环 */
4     while(1) {
5         LED2_TOGGLE; //LED2 翻转
6         LOS_TaskDelay(1000); //1000 个 Tick 延时
7     }
8 }
```

4.4.4 任务挂起与恢复函数

任务的挂起与恢复函数在很多时候都是很有用的，比如想长时间暂停运行某个任务，但是又在需要的时候能够恢复其继续工作，那么是不能删除任务的，因为删除了任务的话，任务的所有的信息都是不可能恢复的。但是可以使用挂起任务函数，仅仅是将任务进入阻塞态，其内部的资源都会保留在任务栈中，同时也不会参与任务的调度，当调用恢复函数的时候，整个任务立即从阻塞态进入就绪态，参与任务的调度，如果该任务的优先级是当前就绪态优先级最高

的任务，那么系统会立即进行一次任务切换，而恢复的任务将按照挂起前的任务状态继续运行，从而达到需要的效果。注意，是继续运行，也就是说，挂起任务之前的任务状态信息，都会被系统保留下来，在恢复后继续按照以前的任务状态运行。挂起任务与恢复任务的函数原型如代码清单 4.8 所示。

代码清单 4.8 挂起与恢复任务函数的原型

```
1 LITE_OS_SEC_TEXT_INIT UINT32 LOS_TaskSuspend(UINT32 uwTaskID)
2
3 LITE_OS_SEC_TEXT_INIT UINT32 LOS_TaskResume(UINT32 uwTaskID)
```

这两个任务函数的使用方法是根据传入的任务 ID 来挂起/恢复对应的任务，任务 ID 是每个任务的唯一标识，本资料提供的例程将通过按键来挂起与恢复 Test1 任务，如代码清单 4.9 加粗部分所示。

代码清单 4.9 任务挂起与恢复的使用实例

```
1 static void Key_Task(void)
2 {
3     UINT8 key;
4     UINT32 uwRet = LOS_OK; /* 定义一个任务的返回类型，初始化为成功的返回值 */
5     /* 任务都是一个无限循环，不能返回 */
6     while(1) {
7         key=KEY_Scan(0);    //得到键值
8         if(key) {
9             switch(key) {
10                 case KEY1_PRES: /* KEY1 被按下 */
11                     printf("挂起 DS0 LED 任务! \r\n");
12                     uwRet = LOS_TaskSuspend(Test1_Task_Handle); /*挂起 Test1 任务*/
13                     if(LOS_OK == uwRet) {
14                         printf("挂起 DS0 LED 任务成功! \r\n");
15                     }
16                     break;
17                 case KEY0_PRES: /* KEY0 被按下 */
18                     printf("恢复 DS0 LED 任务! \r\n");
19                     uwRet = LOS_TaskResume(Test1_Task_Handle); /*恢复 Test1 任务*/
20                     if(LOS_OK == uwRet) {
21                         printf("恢复 DS0 LED 任务成功! \r\n");
22                     }
23                     break;
24             }
25         }
26         LOS_TaskDelay(20); /* 20Ticks 扫描一次 */
27     }
```

4.5 任务的设计要点

作为一个嵌入式开发人员，对自己设计的嵌入式系统要了如指掌，如任务的优先级信息、任务与中断的处理、任务的运行时间、逻辑、状态等，才能设计出好的系统，因此在设计的时候需要根据需求制定框架，并且应该考虑任务运行的上下文环境(中断与任务)、空闲任务以及任务执行时间的合理设计。

(1) 中断服务程序

中断服务程序是一种需要特别注意的上下文环境，它运行在非任务的执行环境下（一般为芯片的一种特殊运行模式），在这个上下文环境中不能使用挂起当前任务的操作，不能有任何阻塞的操作，**在中断中不允许调用带有阻塞机制的 API 函数**。另外需要注意的是，中断服务程序最好保持精简短小，快进快出，一般在中断服务程序中只做标记事件的发生，然后通知任务，让对应的处理任务去执行相关处理，因为**中断的优先级高于系统中任何任务**，如果中断处理时间过长，可能会导致整个系统任务无法正常运行。所以在设计的时候必须考虑中断的频率、中断的处理时间等重要因素，以便配合对应中断事件的处理任务的工作。

(2) 普通任务

任务看似没有什么限制程序执行的因素，似乎所有的操作都可以执行。但是做为一个优先级明确的实时系统，如果一个任务中的程序出现了死循环操作（此处的死循环是指没有阻塞机制的任务循环体），那么比该任务优先级低的任务都将无法执行，当然也包括了空闲任务，因为**没有阻塞的任务不会主动让出 CPU**，低优先级任务是不允许抢占高优先级任务的 CPU 使用权，而高优先级任务可以抢占低优先级任务的 CPU 使用权，这样一来低优先级任务将无法运行，这种情况在实时操作系统中是必须注意的一点，所以在**任务中不允许出现死循环**。如果一个任务只有就绪态而无阻塞态，势必会影响到其他低优先级任务的运行，所以在进行任务设计时，就应该保证任务在不活跃的时候，可以进入阻塞态以让出 CPU 使用权，这就需要设计者明确知道什么情况下让任务进入阻塞态，保证低优先级任务可以正常运行。在实际设计中，一般会需要将需要紧急处理事件的任务优先级设置得高一些。

（3）空闲任务

空闲任务是 LiteOS 系统中没有其他任务运行时自动进入的系统任务。开发者可以选择**空闲时进入低功耗模式**，睡眠模式等。不过需要注意的是，**空闲任务是不允许阻塞也不允许被挂起的，空闲任务是唯一一个不允许出现阻塞情况的任务**，因为 LiteOS 需要保证系统永远都有一个可运行的任务。

（4）任务的执行时间

任务的执行时间一般是指两个方面：一是任务从开始到结束的时间；二是任务的周期。在系统设计的时候这两个时间都需要用户去考虑清楚，例如，系统要求事件 A 对应的处理任务 Task-A 的实时响应指标为 10ms，而 Task-A 的最大运行时间是 1ms，那么 10ms 就是任务 Task-A 的周期了，1ms 则是任务的运行时间，简单来说任务 Task-A 在 10ms 内完成对事件 A 的响应即可。如果此时，系统中还存在另一个以 50ms 为周期的任务 Task-B，它每次运行的最长时间是 100us。在这种情况下，即使把任务 Task-B 的优先级抬到比 Task-A 更高的位置，对系统的实时性指标也没什么影响，因为即使在 Task-A 的运行过程中，Task-B 抢占了 Task-A 的 CPU 使用权，等到 Task-B 执行完毕，消耗的时间也只不过是 100us，通常还是在事件 A 规定的响应时间内(10ms)，Task-A 能够安全完成对事件 A 的响应处理。但是，假如系统中还存在任务 Task-C，其运行时间为 20ms，假如将 Task-C 的优先级设置为比 Task-A 更高，那么在 Task-A 运行的时候，突然间被 Task-C 打断，等到 Task-C 执行完毕，那 Task-A 已经错过对事件 A 的响应了（超过 10ms），这是不允许的。所以在设计的时候，必须考虑任务的时间，**一般来说处理时间更短的任务优先级应设置得更高一些**。

第 5 章 消息队列

5.1 消息队列的基本概念

队列又称消息队列，是操作系统中一种常用于通信的数据结构，队列可以在任务与任务之间、中断和任务之间传送不固定长度的信息。

通过系统提供的消息队列服务函数接口，任务或中断服务例程可以将一条或多条消息放入消息队列中。同样，一个或多个任务也可以从消息队列中获得消息。当有多个消息写入到消息队列时，通常是先进入的消息先传递，也就是说，任务先得到的是最先进入队列的消息，**即先进先出原则**（First In First Out, FIFO），但是**也支持后进先出原则**（Last In First Out, LIFO）。

任务能够从队列中读取消息，当队列中的消息为空时，读取消息的任务将被阻塞，用户可以指定任务的阻塞时间 `uwTimeOut`，在这段时间内，如果队列的消息一直为空，该任务将保持阻塞状态以等待消息到来。当队列中有新消息时，阻塞的任务会被唤醒；当任务等待时间超过了指定的阻塞时间，即使队列中依然没有消息，任务也会自动从阻塞态转为就绪态。

消息队列是一种异步的通信方式，提供了异步处理机制，允许任务将一条消息放入队列，但并不立即处理它，这样队列就起到了缓存消息的作用。

在 LiteOS 中使用消息队列数据结构实现任务间的异步通信，具有如下特性：

- （1）消息以先进先出方式排队，支持异步读写工作方式；
- （2）读队列和写队列都支持超时机制；
- （3）写入消息类型由通信双方约定，可以允许不同长度（不超过消息节点最大值）的任意类型消息；
- （4）消息支持后进先出方式排队（LIFO）；
- （5）**一个任务能够从任意一个消息队列读取和写入消息**；
- （6）**多个任务能够从同一个消息队列读取和写入消息**。

5.2 消息队列的运作机制

创建队列时，根据用户传入队列长度和消息节点大小来开辟相应的内存空间以供该队列使用，初始化消息队列的相关信息，创建成功后返回队列 ID。

在队列控制块中维护一个消息队列头节点位置变量 `usQueueHead`（头指针）和一个尾节点位置变量 `usQueueTail`（尾指针），用于记录当前队列中消息的存储情况。`usQueueHead` 表示队列中存放了消息的节点（后面简称为满节点）的起始位置，`usQueueTail` 表示满节点的结束位置，或者可以理解为队列中未存放消息的节点（空节点）的起始位置，`usQueueHead` 与 `usQueueTail` 之间的节点为存放了消息的满节点。在队列刚创建时 `usQueueHead` 和 `usQueueTail` 均指向队列起始位置。

写队列时，根据 `usQueueTail` 找到满节点末尾后面的第一个空节点作为消息写入区域。如果 `usQueueTail` 已经指向队列结束位置则采用回卷方式，重新回到队列起始位置，**可以将 LiteOS 的消息队列看做是一个首尾相接的环形队列**。根据 `usReadWriteableCnt [OS_QUEUE_WRITE]` 判断队列是否可以写入，**不能对已满的队列进行写队列操作**。

读队列时，根据 `usQueueHead` 找到队列中满节点的起始位置（即当前队列中最早写入消息的节点）进行读取操作。**如果 `usQueueHead` 已经指向队列结束位置也同样采用回卷方式，重新回到队列起始位置**。根据 `usReadWriteableCnt [OS_QUEUE_READ]` 判断队列是否有消息读取，对没有消息的队列进行读队列操作会引起任务挂起（假设用户指定阻塞时间的话）。

删除队列时，根据传入的队列 ID 寻找到对应的队列，把队列状态置为未使用，释放原队列所占的空间，对应的队列控制头置为初始状态。

LiteOS 的消息队列采用两个双向链表来维护，一个链表指向消息队列的头部，一个链表指向消息队列的尾部（`stReadWriteList[QUEUE_HEAD_TAIL]`），通过访问这两个链表就能直接访问对应的消息空间也就是消息队列，并且通过消息队列控制块中的读写类型（`usReadWriteableCnt[QUEUE_READ_WRITE]`）来操作消息队列，消息队列的运作过程如图 5.1 所示。

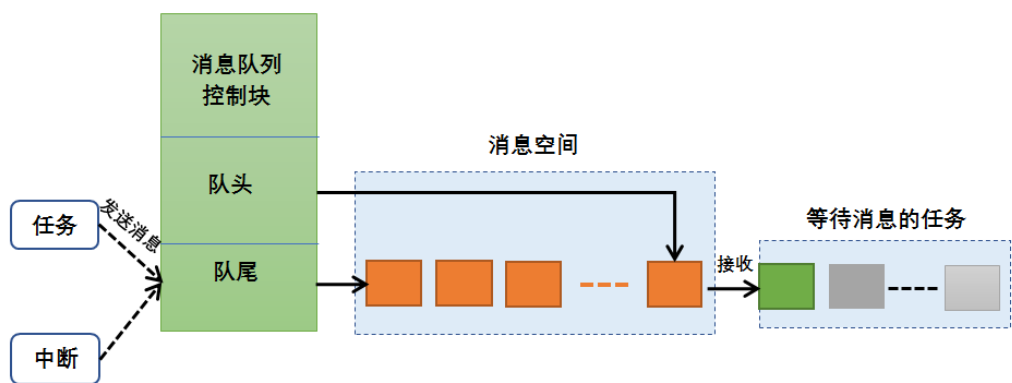


图 5.1 消息队列运作过程

消息队列的读写操作如图 5.2 所示。

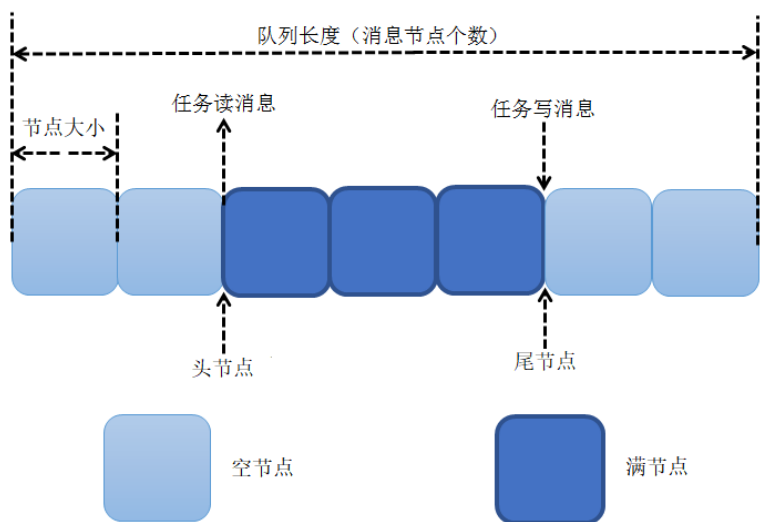


图 5.2 队列读写消息操作示意图

5.3 消息队列的传输机制

既然队列是任务间通信的数据结构，那么它必然可以存储消息数据，消息是存储在消息节点中，而消息节点的大小在创建队列的时候由用户指定。LiteOS 提供的队列是一种先进先出线性表，只允许在一端写入（入队），在另一端进行读取（出队），支持异步读写工作方式，就像来买车票的人一样，先到的人先买到票，后到的人后买到票，不允许插队。当然除此之外，LiteOS 也提供一种**后进先出**的队列操作方式，这种方式**能支持传输紧急的消息**，在某些场合也是会比较常用，就像插队一样，后来买票的人能先买到票。

LiteOS 提供了**两种消息的传输方式**，一种是**复制方式**，另一种是**引用方式**。

所谓复制就是将某个数据直接复制到另一个存储数据的地方，就像在电脑中将某个文件复制到另一个文件中，这两个文件都是一模一样的，修改源文件并不会影响已经复制的文件。而引用传递则是传递数据的指针，该指针指向源数据存储的地址，就好比是从电脑中的源文件创建了一个文件的快捷方式，通过快捷方式也能打开源文件，并且快捷方式占用的存储空间是非常小的。但是该方式有个缺点，假如修改了源文件的内容，那么通过快捷方式打开的文件，其内容也会相应被修改，这样就造成数据的可变性，在某些场合下是不安全的。

用户可以根据消息的大小与重要性来选择消息的传递方式，如果消息很重要，选择复制的方式会更加安全，如果消息的数据量很小，也可以选择复制的方式；如果消息只是一些不重要的内容或者消息数据量很大，则可以选择引用方式。

5.4 消息队列的阻塞机制

在系统中创建了一个队列，每个任务都可以去对它进行读写操作，但是为了保护每个任务对它进行读写操作的过程，则必须要有阻塞机制，在某个任务对它读写操作的时候，必须保证该任务能正常完成读写操作，而不受后来的任务干扰。除此之外，当队列已满的时候，其他任务就不能将消息写入而导致消息的覆盖，同理，当队列为空的时候，读取消息的任务也无法读取消息，这种机制可以称之为阻塞机制。

5.4.1 出队阻塞

假设有一个任务 A 对某个队列进行读操作的时候（也就是出队），发现它没有消息，那么此时任务 A 有三个选择：第一个选择，任务 A 不等待消息的到来，这样任务 A 不会进入阻塞态；第二个选择，任务 A 阻塞等待，等待时间由用户指定，比如可以是 1000 个 Tick，在超时时间到来之前，如果队列有消息了，那任务 A 恢复就绪态，读取队列的消息，如果任务 A 刚好是最高优先级的就绪任务，那么系统将进行一次任务调度。如果等待时间已经超出指定时间，队列还没有消息可以读取，那么任务 A 将恢复为就绪态；第三个选择，任务 A 进入阻塞态一直等待消息的到来，直到完成读取队列的消息。

5.4.2 入队阻塞

同理，对某个队列的写操作也是一样的（写操作就是将消息写入队列中，也就是入队），当任务 A 向某个队列中写入一个消息时，发现这个队列已经满了，队列中无可用的消息节点，LiteOS 出于对队列中消息的保护，不允许新的消息写入，这样一来任务的写操作就会被阻塞，任务的阻塞时间由用户指定。在指定的阻塞时间内如果还不能完成入队操作，写入消息的任务会收到一个错误代码 `LOS_ERRNO_QUEUE_ISFULL`，然后解除阻塞状态。注意，只有在任务中写入消息才允许进行阻塞状态，而在中断中写入消息是不允许有阻塞的，用户必须将阻塞时间设置为 0，否则会直接返回错误代码 `LOS_ERRNO_QUEUE_WRITE_IN_INTERRUPT`，因为在中断中不允许出现阻塞的情况。

假如有多个任务阻塞在一个消息队列中，那么这些阻塞的任务将按照任务优先级进行排序，优先级高的任务将优先获得队列的访问权。

5.5 常用的消息队列操作函数

使用消息队列的典型流程如下。

- （1）创建消息队列 `LOS_QueueCreate()`;
- （2）创建成功后，可以得到消息队列的 ID 值;
- （3）写队列操作函数 `LOS_QueueWrite()`;
- （4）读队列操作函数 `LOS_QueueRead()`;
- （5）删除队列 `LOS_QueueDelete()`。

5.5.1 消息队列创建函数 `LOS_QueueCreate()`

消息队列创建函数 `LOS_QueueCreate()` 用于创建一个队列，读者可以根据自己的需要去创建队列，可以指定队列的长度以及消息节点的大小等信息，LiteOS 创建队列的函数原型如代码清单 5.1 所示。

创建消息队列时系统会先给消息队列分配一块内存空间，这块内存的大小等于“单个消息节点大小+`sizeof(UINT32)`（4 个字节）”与消息队列长度的乘

积，接着再初始化消息队列，此时消息队列为空。LiteOS 的消息队列控制块由多个元素组成，当系统初始化时，系统会为控制块分配对应的内存空间，用于保存消息队列的基本信息如消息的存储位置，头指针 `usQueueHead`、尾指针 `usQueueTail`、消息大小 `usQueueSize` 以及队列长度 `usQueueLen` 等。在消息队列创建成功的时候，这些内存就被分配占用了，只有删除消息队列的时候，这段内存才会被释放掉，创建成功的队列已经确定队列的长度与消息节点的大小，且无法再次更改，每个消息节点可以存放不大于消息大小 `usQueueSize` 的任意类型的消息，消息节点个数的总和就是队列的长度。

代码清单 5.1 队列创建函数 `LOS_QueueCreate()`函数原型

```
1 LITE_OS_SEC_TEXT_INIT UINT32 LOS_QueueCreate (CHAR *pcQueueName,      (1)
2                                     UINT16 usLen,                      (2)
3                                     UINT32 *puwQueueID,                (3)
4                                     UINT32 uwFlags,                    (4)
5                                     UINT16 usMaxMsgSize);              (5)
```

代码清单 5.1(1): `pcQueueName` 是消息队列名称，LiteOS 保留，暂时未使用。

代码清单 5.1(2): `usLen` 是队列长度，值范围是 1~0xFFFF。

代码清单 5.1(3): `puwQueueID` 是消息队列 ID 变量指针，该变量用于保存创建队列成功时返回的消息队列 ID，由用户定义消息队列 ID 变量，对消息队列的读写操作都是通过消息队列 ID 来实现的。

代码清单 5.1(4): `uwFlags` 是队列模式，保留参数，暂不使用。

代码清单 5.1(5): `usMaxMsgSize` 是消息节点大小（单位为字节），其取值范围为 1~(0xFFFF-4)。

队列控制块与任务控制块类似，每一个队列都由对应的队列控制块维护，队列控制块中包含了队列的所有信息，比如队列的一些状态信息，使用情况等，如代码清单 5.2 所示。

代码清单 5.2 队列控制块

```
1 typedef struct tagQueueCB{
2     UINT8 *pucQueue;      /* 队列指针 */
3     UINT16 usQueueState; /* 队列状态 */
4     UINT16 usQueueLen;    /* 队列中消息个数 */
5     UINT16 usQueueSize;   /* 消息节点大小 */
6     UINT16 usQueueID;     /* 队列 ID */
7     UINT16 usQueueHead;   /* 消息头节点位置（数组下标）*/
```

```

8  UINT16 usQueueTail;    /* 消息尾节点位置（数组下标）*/
9  UINT16 usReadWriteableCnt[2]; /*可读或可写资源的计数, 0:可读, 1:可写*/
10 LOS_DL_LIST stReadWriteList[2]; /*指向要读取或写入的链表的指针, 0:读列表, 1:写列表/
11 LOS_DL_LIST stMemList; /* ** <指向内存链表的指针* /
12 }QUEUE_CB_S;

```

创建队列必须是调用 `LOS_QueueCreate()` 函数进行创建, 在创建成功后返回一个队列 ID。在创建队列时会返回创建的情况, 如果返回 `LOS_OK`, 则表明队列创建成功, 否则返回其他错误代码。创建消息队列的应用实例如代码清单 5.3 加粗部分所示。

代码清单 5.3 队列创建函数 `LOS_QueueCreate()` 实例

```

1  UINT32 uwRet = LOS_OK; /* 定义一个创建队列的返回类型, 初始化为创建成功的返回值 */
2
3  /* 创建一个测试队列*/
4  uwRet = LOS_QueueCreate("Test_Queue", /* 队列的名称, 保留, 未使用*/
5                               128,      /* 队列的长度 */
6                               &Test_Queue_Handle, /* 队列的 ID(句柄) */
7                               0,          /* 队列模式, 官方暂时未使用 */
8                               16);        /* 最大消息大小(字节数)*/
9  if(uwRet != LOS_OK)
10 {
11     printf("Test_Queue 队列创建失败! \n");
12 }

```

5.5.2 消息队列删除函数 `LOS_QueueDelete()`

队列删除函数是根据队列 ID 直接删除的, 删除之后这个队列的所有信息都会被系统回收清空, 而且不能再次使用这个队列了, 但是需要注意的是, 队列在使用或者阻塞中是不能被删除的, 如果某个队列没有被创建, 那也是无法被删除的, `uwQueueID` 是 `LOS_QueueDelete()` 函数传入的参数, 是队列 ID, 表示要删除队列, 其函数原型如代码清单 5.4 所示。

代码清单 5.4 `LOS_QueueDelete` 函数原型

```

1  LITE_OS_SEC_TEXT_INIT UINT32 LOS_QueueDelete(UINT32 uwQueueID)

```

队列删除函数的实例如代码清单 5.5 加粗部分所示, 如果队列删除成功, 则返回 `LOS_OK`, 否则返回其他错误代码。

代码清单 5.5 LOS_QueueDelete()函数使用实例

```

1  UINT32 uwRet = LOS_OK; /* 定义一个删除队列的返回类型，初始化为删除成功的返回值 */
2
3  uwRet = LOS_QueueDelete(Test_Queue_Handle); /* 删除队列 */
4  if(uwRet != LOS_OK) /* 删除队列失败，返回其他错误代码 */
5  {
6      printf("删除队列失败! \n");
7  }
8  else /* 删除队列成功，返回 LOS_OK */
9  {
10     printf("删除队列成功! \n");
11 }

```

5.5.3 消息队列写消息函数

1、不带复制方式写入 LOS_QueueWrite()

任务或者中断服务程序都可以向消息队列写入消息，当写入消息时，如果队列未满，LiteOS 会将消息添加到队列中当前尾节点的后面，否则，会根据用户指定的阻塞时间进行阻塞，在这段时间中，如果队列还是满的，该任务将保持阻塞状态以等待队列有空节点。如果系统中有任务从其等待的队列中读取了消息，该队列转为未满，任务将自动由阻塞态转为就绪态。**当任务等待的时间超过了指定的阻塞时间，即使队列中还是满的，任务也会自动从阻塞态变成就绪态**，此时写入消息的任务或者中断程序（中断程序不会阻塞）会收到一个错误代码 LOS_ERRNO_QUEUE_ISFULL。

LiteOS 也支持后进先出（LIFO）方式写入消息，即支持写入紧急消息，写入紧急消息的过程与写入普通消息几乎一样，唯一的**不同是，当写入紧急消息时，写入的位置是队列中当前头节点的前面而非尾节点的后面**，这样读取任务就能够优先读取到紧急消息，从而及时进行消息处理。

LiteOS 消息队列的传递方式有两种：一种是不带复制传递消息；另一种是带复制传递消息，不带复制传递消息的函数原型如代码清单 5.6 所示，其应用实例如代码清单 5.7 加粗部分所示。

代码清单 5.6 LOS_QueueWrite()函数原型

1	LITE_OS_SEC_TEXT	UINT32	LOS_QueueWrite(UINT32 uwQueueID,	(1)
2			VOID *pBufferAddr,	(2)
3			UINT32 uwBufferSize,	(3)
4			UINT32 uwTimeOut)	(4)

代码清单 5.6(1): uwQueueID 是队列 ID, 由 LOS_QueueCreate()函数返回, 其值范围为 1~LOSCFG_BASE_IPC_QUEUE_LIMIT (config_target.h 中配置的消息队列个数上限值)。

代码清单 5.6(2): pBufferAddr 为写入消息的起始地址。

代码清单 5.6(3): uwBufferSize 为写入消息的大小。

代码清单 5.6(4): uwTimeOut 是等待时间, 也就是由用户指定的阻塞时间, 其值范围为 0~LOS_WAIT_FOREVER, 单位为 Tick, 当 uwTimeOut 为 0 的时候是不等待, 为 LOS_WAIT_FOREVER 时候是一直等待, 如果在中断中使用该函数, uwTimeOut 的值必须为 0。

代码清单 5.7 LOS_QueueWrite()函数应用实例

```
1  UINT32 send_data1 = 1; /* 写入队列的第一个消息 */
2  UINT32 send_data2 = 2; /* 写入队列的第二个消息 */
3  static void Key_Task(void)
4  {
5      UINT8 key;
6      UINT32 uwRet = LOS_OK; /* 定义一个任务的返回类型, 初始化为成功的返回值 */
7      /* 任务都是一个无限循环, 不能返回 */
8      while(1) {
9          key=KEY_Scan(0);    //得到键值
10         if(key) {
11             switch(key) {
12                 case KEY1_PRES: /* KEY1 被按下 */
13                     uwRet = LOS_QueueWrite(Test_Queue_Handle, /* 写入的队列 ID */
14                     &send_data1, /* 写入的消息 */
15                     sizeof(send_data1), /* 消息的大小 */
16                     0); /* 等待时间为 0 */
17                     if(LOS_OK != uwRet) {
18                         printf("消息不能写入到消息队列! 错误代码 0x%x \r\n", uwRet);
19                     }
20                     break;
21                 case KEY0_PRES: /* KEY0 被按下 */
22                     uwRet = LOS_QueueWrite(Test_Queue_Handle, /* 写入的队列 ID */
23                     &send_data2, /* 写入的消息 */
24                     sizeof(send_data2), /* 消息的长度 */
25                     0); /* 等待时间为 0 */
26                     if(LOS_OK != uwRet) {
27                         printf("消息不能写入到消息队列! 错误代码 0x%x \r\n", uwRet);
28                     }
29                     break;
30             }
31         }
32     }
```



```

30     }
31     LOS_TaskDelay(20); /* 20Ticks 扫描一次 */
32 }
33 }

```

写入队列按照 LiteOS 的 API 进行操作即可，但是有几个点需要注意：

- (1) 在写队列的操作前应先创建要写入的队列；
- (2) 在中断服务程序中，必须使用非阻塞模式写入，也就是等待时间（指定的阻塞时间）为 0 个 Tick；
- (3) 在初始化 LiteOS 之前不能调用此 API；
- (4) 写入消息的大小不能大于消息节点的大小（字节数）；
- (5) 写入队列节点中的是消息的地址。

2、带复制写入 LOS_QueueWriteCopy()

LOS_QueueWriteCopy()是带复制写入的函数接口，函数原型如代码清单 5.8 所示，其应用实例如代码清单 5.9 加粗部分所示。

代码清单 5.8 LOS_QueueWriteCopy()函数原型

```

1 LITE_OS_SEC_TEXT UINT32 LOS_QueueWriteCopy(UINT32 uwQueueID,      (1)
2                                     VOID *pBufferAddr,             (2)
3                                     UINT32 uwBufferSize,           (3)
4                                     UINT32 uwTimeOut)               (4)

```

代码清单 5.8(1): uwQueueID 是由 LOS_QueueCreate 创建的队列 ID，其值范围为 1~LOSCFG_BASE_IPC_QUEUE_LIMIT（config_target.h 中配置的消息队列个数上限值）。

代码清单 5.8(2): pBufferAddr 为写入消息的起始地址，起始地址不能为空。

代码清单 5.8(3): uwBufferSize 是指定写入消息的大小，其值不能大于消息节点大小。

代码清单 5.8(4): uwTimeOut 是等待时间（由用户指定的阻塞时间），其值范围为 0~LOS_WAIT_FOREVER，单位为 Tick，当 uwTimeOut 为 0 的时候是不等待，为 LOS_WAIT_FOREVER 时候是一直等待。

代码清单 5.9 LOS_QueueWriteCopy()函数应用实例

```

1 UINT32 send_data1 = 1; /* 写入队列的第一个消息 */
2 UINT32 send_data2 = 2; /* 写入队列的第二个消息 */
3 static void Send_Task(void)
4 {

```

```

5  UINT8 key;
6  UINT32 uwRet = LOS_OK; /* 定义一个任务的返回类型，初始化为成功的返回值 */
7  /* 任务都是一个无限循环，不能返回 */
8  while(1) {
9      key=KEY_Scan(0);    //得到键值
10     if(key) {
11         switch(key) {
12             case KEY1_PRES: /* KEY1 被按下 */
13                 uwRet = LOS_QueueWriteCopy(Test_Queue_Handle, /*写入的队列 ID*/
14                 &send_data1, /* 写入的消息 */
15                 sizeof(send_data1), /* 消息的大小 */
16                 0); /* 等待时间为 0 */
17                 if(LOS_OK != uwRet){
18                     printf("消息不能写入到消息队列！错误代码 0x%x \r\n",uwRet);
19                 }
20                 break;
21             case KEY0_PRES: /* KEY0 被按下 */
22                 uwRet = LOS_QueueWriteCopy(Test_Queue_Handle, /*写入的队列 ID*/
23                 &send_data2, /* 写入的消息 */
24                 sizeof(send_data2), /* 消息的长度 */
25                 0); /* 等待时间为 0 */
26                 if(LOS_OK != uwRet){
27                     printf("消息不能写入到消息队列！错误代码 0x%x \r\n",uwRet);
28                 }
29                 break;
30             }
31         LOS_TaskDelay(20); /* 20Ticks 扫描一次 */
32     }
33 }

```

带复制写入操作有几点需要注意的地方：

- (1) 在写队列的操作前应先创建要写入的队列；
- (2) 在中断服务程序中，必须使用非阻塞模式写入，也就是等待时间为 0 个 Tick。
- (3) 在初始化 LiteOS 之前不能调用此 API。
- (4) 写入消息的大小不能大于消息节点的大小（字节数）；
- (5) 写入队列节点中的是存储在*pBufferAddr 中的消息。

5.5.4 消息队列读消息函数

1、不带复制方式读取 LOS_QueueRead()

消息队列的传输方式分为两种：一种是不带复制的；另一种是带复制的。不带复制读取消息函数原型如代码清单 5.10 所示。该函数用于读取指定队列中的消息，并将获取的消息存储到 pBufferAddr 指定的地址，用户需要指定读取消息的存储地址与大小，该函数的应用实例如代码清单 5.11 加粗部分所示。

代码清单 5.10 LOS_QueueRead()函数原型

```
1 LITE_OS_SEC_TEXT UINT32 LOS_QueueRead(UINT32 uwQueueID, (1)
2                                     VOID *pBufferAddr, (2)
3                                     UINT32 uwBufferSize, (3)
4                                     UINT32 uwTimeOut) (4)
```

代码清单 5.10(1): uwQueueID 是由 LOS_QueueCreate 创建的队列 ID，其值范围为 1~LOSCFG_BASE_IPC_QUEUE_LIMIT。

代码清单 5.10(2): pBufferAddr 是存放读出消息的缓冲区（指定的一块存储区域）起始地址。

代码清单 5.10(3): uwBufferSize 是存放读出消息的缓冲区大小（字节数），该值不能小于消息节点大小，否则会报错。

代码清单 5.10(4): uwTimeOut 是等待时间（指定的阻塞时间），其值范围为 0~LOS_WAIT_FOREVER，单位为 Tick，当 uwTimeOut 为 0 的时候是不等待，为 LOS_WAIT_FOREVER 时候是一直等待。

代码清单 5.11 LOS_QueueRead()函数应用实例

```
1 static void Receive_Task(void)
2 {
3     UINT32 uwRet = LOS_OK;
4     UINT32 r_queue; /* r_queue 用于存储读出的消息（读出消息缓冲区） */
5     UINT32 buffsize = 16; /* 读出消息缓冲区的大小不能小于设置的消息节点大小(字节数) */
6     while(1){
7     /* 队列读取，等待时间为一直等待 */
8         uwRet = LOS_QueueRead(Test_Queue_Handle, /* 读取队列的 ID */
9                               &r_queue, /* 读出消息的存储缓冲区地址 */
10                              buffsize, /* 读出消息的存储缓冲区大小 */
11                              LOS_WAIT_FOREVER); /* 一直等待 */
12         if(LOS_OK == uwRet){
13             printf("本次读取到的消息是%d \r\n", *(UINT32 *)r_queue );
```

```

14     }
15     else{
16         printf("消息读取出错, 错误代码 0x%X \r\n", uwRet);
17     }
18     LOS_TaskDelay(20);
19 }
20 }

```

读取消息的时候需要注意以下几点：

（1）使用 `LOS_QueueRead()` 这个函数之前应先创建需要读取消息的队列，并根据队列 ID 进行读取消息。

（2）**队列读取采用的是先进先出（FIFO）模式**，读出的消息是当前队列中最先写入的消息。

（3）用户必须要定义一个存放读出消息的存储区域（或存储单元）作为缓冲区，并把缓冲区的起始地址传递给 `LOS_QueueRead()` 函数，否则，将发生地址非法的错误。

（4）在**中断服务程序中，必须使用非阻塞模式读取消息，也就是等待时间为 0 个 Tick。**

（5）在初始化 LiteOS 之前不能调用此 API。

（6）由于采用不带复制方式读取，读出消息的存储缓冲区（`r_queue` 变量）中存放的是消息内容的地址，而不是消息内容本身。

（7）`LOS_QueueReadCopy()` 和 `LOS_QueueWriteCopy()` 是一组接口，`LOS_QueueRead()` 和 `LOS_QueueWrite()` 是一组接口，两组接口需要配套使用。

2、带复制读取 `LOS_QueueReadCopy()`

`LOS_QueueReadCopy()` 是带复制读取消息函数，其函数原型如代码清单 5.12 所示，应用实例如代码清单 5.13 加粗部分所示。

代码清单 5.12 `LOS_QueueReadCopy()` 函数原型

1	<code>LITE_OS_SEC_TEXT UINT32 LOS_QueueReadCopy(UINT32 uwQueueID,</code>	(1)
2	<code>VOID *pBufferAddr,</code>	(2)
3	<code>UINT32 *puwBufferSize,</code>	(3)
4	<code>UINT32 uwTimeOut)</code>	(4)

代码清单 5.12(1): `uwQueueID` 是由 `LOS_QueueCreate` 创建的队列 ID，其值范围为 `1~LOS_CFG_BASE_IPC_QUEUE_LIMIT`。

代码清单 5.12(2): pBufferAddr 是存放读出消息的缓冲区起始地址。

代码清单 5.12(3): uwBufferSize 是存放读出消息的缓冲区大小, 该值不能小于消息节点大小。

代码清单 5.12(4): uwTimeOut 是等待时间, 其值范围为 0~LOS_WAIT_FOREVER, 单位为 Tick, 当 uwTimeOut 为 0 的时候表示不等待, 为 LOS_WAIT_FOREVER 的时候表示一直等待。

代码清单 5.13 LOS_QueueReadCopy()函数应用实例

```
1 static void Receive_Task(void)
2 {
3  /* 定义一个返回类型变量, 初始化为 LOS_OK */
4  UINT32 uwRet = LOS_OK;
5  UINT32 r_queue;
6  UINT32 buffsize = 10;
7  /* 任务都是一个无限循环, 不能返回 */
8  while (1)
9  {
10     buffsize = 16; //更新传递进来的 buffsize 大小
11     /* 队列读取, 等待时间为一直等待 */
12     uwRet = LOS_QueueReadCopy(Test_Queue_Handle,
13                               &r_queue, /* 读出消息的存储缓冲区地址 */
14                               &bffsize, /* 读出消息的存储缓冲区大小 */
15                               LOS_WAIT_FOREVER); /* 一直等待 */
16
17     if(LOS_OK == uwRet)
18     {
19         printf("本次读取到的消息是%d\r\n", r_queue); /*注意与不带复制读取的区别*/
20     }
21     else
22     {
23         printf("消息读取出错, 错误代码 0x%X\r\n", uwRet);
24     }
25 }
26 }
```

LOS_QueueReadCopy()函数需要注意以下几点。

(1) 使用 LOS_QueueReadCopy()这个函数之前应先创建需要读取消息的队列, 并根据队列 ID 进行读取消息。

(2) 队列读取采用的是先进先出 (FIFO) 模式, 读出的消息是当前队列中最先写入的消息。

(3) 用户必须要定义一个存放读出消息的存储区域（或存储单元）作为缓冲区，并把缓冲区的起始地址传递给 `LOS_QueueReadCopy()` 函数，否则，将发生地址非法的错误。

(4) 除了中断，一般不要在非阻塞模式下对队列进行读写操作，中断服务程序通常不读取消息，如果要读取消息，需要设为非阻塞模式，也就是等待时间为 0 个 Tick。

(5) 在初始化 LiteOS 之前不能调用此 API。

(6) 由于采用带复制方式读取，读出消息的存储缓冲区（`r_queue` 变量）中存放的是消息内容本身，而非消息内容的地址，因此该缓冲区必须足够大。

(7) 用户必须在读取消息时指定读出消息的存储缓冲区大小，其值不能小于消息节点大小。如本例中的 `buffsize`，该变量既作为输入又作为输出，作为输入是指定缓冲区的大小；作为输出是用于保存读取到消息的大小，把读取到的消息大小写在 `buffsize` 变量中，在调用 `LOS_QueueReadCopy()` 函数前应该注意更新 `buffsize` 的值，使其与消息节点大小相匹配。

第 6 章 信号量

6.1 信号量的基本概念

信号量（Semaphore）是一种实现任务间通信的机制，实现任务之间同步或临界资源的互斥访问，常用于不同任务间工作的协作，或协助一组相互竞争的任务来访问临界资源。在多任务系统中，各任务之间需要同步或互斥实现临界资源的保护，信号量功能可以为用户提供这方面的支持。比如可用信号量标记某个事件的状态，不同的任务可根据事件的状态进行相应的处理，从而实现任务间的同步与协作。

抽象来说，信号量是一个非负整数，所有获取它的任务都会将该整数减一，当该整数值为零时将无法被获取，所有试图获取它的任务都将进入阻塞态。通常一个信号量的计数值用于对应有效的资源数，表示剩下的可被占用的互斥资源数，其值的含义分两种情况：

（1）0：表示没有积累下来的释放信号量操作，且可能有任务阻塞在此信号量上；

（2）正值：表示有一个或多个释放信号量操作。

6.1.1 二值信号量

二值信号量既可以用于同步功能也可以用于临界资源访问保护。

二值信号量和互斥锁非常相似，但是有一些细微差别：互斥锁有优先级继承机制，而二值信号量则没有这个机制。这使得二值信号量更偏向应用于同步功能（任务与任务间的同步或任务和中断间同步），而互斥锁更偏向应用于临界资源的访问。除此之外，互斥锁有所有者属性，只有持有锁的任务才能获取/释放，而非持有者、中断不可以释放；信号量则是任何任务包括中断都可以释放。互斥锁的获取（上锁）、释放（解锁）必须配对。而信号量则不必配对，更适合类似生产者与消费者间交互的场景，生产者为特定的任务、中断；消费者则为其他的任务，一般不会配对，生产者只能释放（post），消费者只能获

取（pend）。同时互斥锁因为有所有者属性，所以可以进行嵌套，持有锁的任务在获取同一把锁时不会被阻塞，而信号量如果用来做互斥保护，出现嵌套情况则会产生死锁。

用作同步时，信号量在创建后应被置为空，任务 1 获取信号量而进入阻塞态，任务 2 在满足某种条件后，释放信号量，于是任务 1 获得信号量从而恢复就绪态，参与系统的调度，从而达到了两个任务间的同步。同样，信号量允许在中断服务函数中释放，如此一来任务 1 也会得到信号量，从而达到任务与中断间的同步。

中断服务函数应该快进快出，处理的时间不能过长，在裸机开发中常用的做法是在中断中做一个标记，然后在主程序中轮询判断，当标记事件发生了，再做相应的处理。这种轮询的处理方式是难以实现实时处理的，比如：当一个事件发生了，但是主程序的循环中还有很多函数尚未处理，那么 CPU 必须等到处理完所有的函数才能去处理触发的事件，这就是裸机开发中的“实时响应，轮询处理”，而信号量则不同，在释放信号量的时候能立即将任务转变为就绪态，如果任务的优先级在就绪任务中是最高的，任务就能立即被执行，这就是操作系统中的“实时响应，实时处理”。在 LiteOS 中使用信号量用于任务与任务之间、中断与任务之间的同步，可以大大提高事件处理的效率和实时性。

信号量以同步为目的和以互斥为目的在使用时有以下不同：

（1）用作互斥时，信号量创建后，信号量中可用信号量个数是 1，在需要使用临界资源时，先取信号量，使其变空（0），这样其他任务需要使用临界资源时就会因为无法取到信号量而阻塞，从而保证了临界资源的安全，当任务使用完临界资源后必须要及时释放信号量。在实际应用中一般不会使用信号量作互斥。

（2）用作同步时，信号量在创建后被置为空，任务 1 取信号量而阻塞，任务 2 在满足某种条件后（如检测到某个事件发生后）释放信号量，于是任务 1 得以进入就绪态或运行态（如果就绪任务的优先级是最高的，那么会立即被运行），从而达到了两个任务间的同步。

6.1.2 计数信号量

顾名思义，计数信号量是用于计数的，在实际的使用中，计数信号量常用于事件计数与资源管理。每当某个事件发生时，任务/中断释放一个信号量（信

号量计数值加 1)，当处理该事件时（一般在任务中处理），处理任务会取走一个信号量（信号量计数值减 1），信号量的计数值则表示还剩余多少个事件没被处理。此外，系统还有很多资源，也可以使用计数信号量进行资源管理，信号量的计数值表示系统中可用的资源数目，任务必须先获取到信号量才能访问资源，当信号量的计数值为零时表示系统没有可用的资源，但是要注意，在使用完资源的时候必须及时归还信号量，否则当计数值为 0 的时候任务就无法访问该资源了。

计数信号量允许多个任务对其进行操作，但限制了任务的数量。比如有一个停车场，里面只有 100 个车位，那么只能停 100 辆车，也相当于有 100 个信号量。假设一开始停车场的车位有 100 个，那么每进去一辆车就要消耗一个停车位，车位的数量就要减一。相应的，信号量在获取之后也需要减一，当停车场停满了 100 辆车的时候，此时的停车位为 0，其他车就不能停进去了，否则将造成事故。同理当信号量的值为 0，其他任务就无法获取到信号量。当有车从停车场离开的时候，车位又有空余出来的，那么其他车就可以进入停车场了。信号量的操作也是一样的，当有任务释放了信号量，其他任务才能对这个资源进行访问。

6.2 二值信号量的应用场景

在嵌入式操作系统中二值信号量是任务与任务间、中断与任务间同步的重要手段。信号量分为二值信号量与计数信号量，什么是二值信号量呢？当信号量被获取时其值为 0，当信号量被释放时其值为 1，把这种只有 0 和 1 两种情况的信号量称为二值信号量。

在多任务系统中经常会使用二值信号量，比如，有一个任务需要等待某个中断或者事件发生了，再去执行对应的处理，那么该任务可以处于阻塞态等待信号量，直到中断或者事件发生后释放信号量，该任务才被唤醒去执行相应的处理。

当事件处理任务在运行中获取信号量时，如果指定事件尚未发生，信号量为空（0），则会进入阻塞状态。事件发生后，相应的任务/中断便会释放信号量，事件处理任务取得信号量便被唤醒去执行对应的操作，任务执行完毕不需要归还信号量，这样可以大大提高 CPU 的执行效率，以及事件响应的实时性。

二值信号量在任务与任务间同步的应用场景：假设有一个温湿度采集显示系统，包括温湿度采集任务和液晶显示任务，温湿度采集任务每秒钟采集一次数据，在完成数据采集后，使用信号量进行一次同步，告诉液晶显示任务可以更新数据了，那么液晶屏只需要在每秒钟温湿度数据更新的时候刷新一次即可，这样不但在每次数据变化的时候都能及时更新显示数据，而且也不会因为不必要的液晶显示刷新操作造成 CPU 资源的浪费。

同理，二值信号量在任务与中断间同步的应用场景：假设一个系统通过串口接收数据并进行处理，那么可以采用中断接收数据，任务处理数据，通过二值信号量同步任务与中断的方式。任务获取信号量的时候进入阻塞态，当中断接收到数据时，释放一个二值信号量，任务获取到信号量后将立即从阻塞态转入就绪态，然后执行相应的处理。这样既避免 CPU 资源的浪费，也提高了接收数据处理的实时性。

6.3 二值信号量的运作机制

在系统初始化时会进行信号量初始化，为配置的 `LOSCFG_BASE_IPC_SEM_LIMIT` 个信号量申请内存（该宏定义的值在 `target_config.h` 中指定），并把所有的信号量初始化为未使用，并加入到信号量未使用链表 `g_stUnusedSemList` 中供系统使用。

信号量创建时，从未使用的信号量链表中获取一个信号量资源，该信号量的最大计数值为 1（`OS_SEM_BINARY_MAX_COUNT`），也就是二值信号量（计数值 0，1）。

任何任务都可以从已创建的二值信号量资源中获取一个二值信号量，若当前信号量有效，则获取成功并且返回 `LOS_OK`，否则任务会根据用户指定的阻塞时间等待其他任务/中断释放信号量，在这段时间中，系统将任务处于阻塞态，挂到该信号量的阻塞等待列表中，示意图如图 6.1 所示。

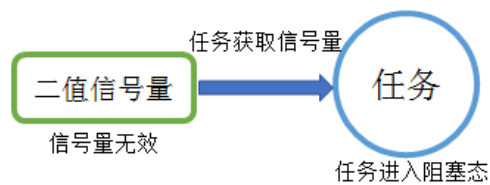


图 6.1 获取信号量无效

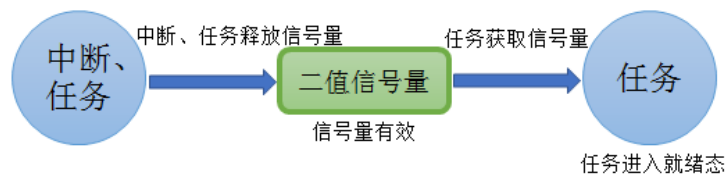


图 6.2 二值信号量运作机制

假如中断或者另一个任务释放一个二值信号量，那么处于阻塞态的任务将恢复为就绪态，其过程如图 6.2 所示。

6.4 计数信号量的运作机制

计数信号量可以用于资源管理，允许多个任务获取信号量访问共享资源，但会限制任务的最大数目。访问的任务数达到可支持的最大数目时，会阻塞其他试图获取该信号量的任务，直到有任务释放了信号量，这就是计数信号量的运作机制。比如某个资源限定只能有 3 个任务访问，那么任务 4 访问的时候，会因为获取不到信号量而进入阻塞，当某个任务（比如任务 1）释放该资源时，任务 4 才能获取到信号量从而进行资源的访问，其运作的机制如图 6.3 所示。

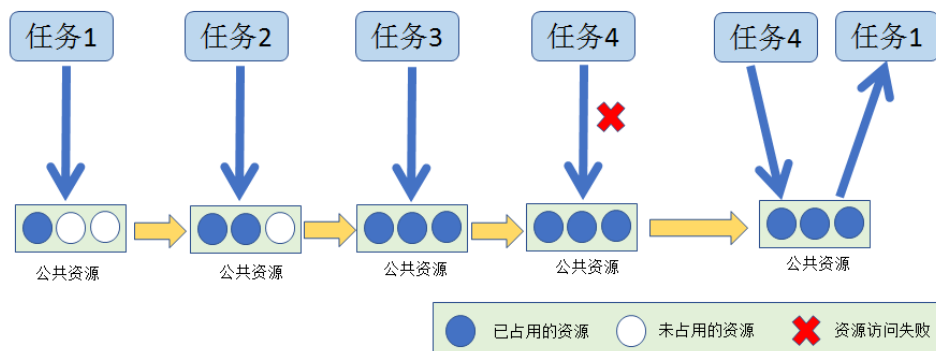


图 6.3 计数信号量运作示意图

6.5 信号量的使用

6.5.1 信号量控制块

信号量控制块与任务控制类似，系统中每个信号量都有对应的信号量控制块，信号量控制块中包含了信号量的所有信息，比如信号量的状态信息，使用情况、以及信号量阻塞列表等，如代码清单 6.1 所示。

代码清单 6-1 信号量控制块

```
1 typedef struct {
2     UINT16 usSemStat;           (1)
3     UINT16 usSemCount;         (2)
4     UINT16 usMaxSemCount;      (3)
5     UINT16 usSemID;            (4)
6     LOS_DL_LIST stSemList;     (5)
7 }SEM_CB_S;
```

代码清单 6.1(1): usSemStat 表示信号量状态，标志信号量是否被使用。

代码清单 6.1(2): usSemCount 表示可用信号量的个数。

代码清单 6.1(3): usMaxSemCount 表示可用信号量的最大容量，在二值信号量中，其值为 OS_SEM_BINARY_MAX_COUNT，也就是 1；而在计数信号量中，它的最大值是 OS_SEM_COUNTING_MAX_COUNT，也就是 0xFFFF。

代码清单 6.1(4): usSemID 表示信号量 ID。

代码清单 6.1(5): stSemList 是信号量阻塞列表，用于记录正在等待信号量的任务。

6.5.2 二值信号量创建函数 LOS_BinarySemCreate()

LiteOS 提供的二值信号量创建函数是 LOS_BinarySemCreate()，因为创建的是二值的信号量，所以该信号量的容量只有一个，里面要么是满，要么是空，在创建的时候用户可以自己定义它初始可用信号量的个数，范围是 0~ 1，LOS_BinarySemCreate()函数原型如代码清单 6.2 所示。

代码清单 6.2 LOS_BinarySemCreate()函数原型

```
1 LITE_OS_SEC_TEXT_INIT UINT32 LOS_BinarySemCreate(UINT16 usCount, UINT32
*puwSemHandle)
```

在创建信号量的时候,只需要传入用户定义的保存二值信号量 ID 的变量地址与初始化可用信号量个数的值即可,对于二值信号量可以指定初始信号量个数为 1 或者为 0。如果指定信号量有效个数为 1,则表明信号量是有效的,任务可以立即获取信号量。如果不需要立即获取信号量的情况下,可以将信号量可用个数的值初始化为 0,在事件发生后再由中断或其他任务释放信号量。二值信号量创建函数应用实例如代码清单 6.3 加粗部分所示。

代码清单 6.3 LOS_BinarySemCreate()函数应用实例

```
1 UINT32 uwRet = LOS_OK; /* 定义一个创建的返回类型,初始化为创建成功的返回值 */
2
3 /* 创建一个二值信号量 */
4 uwRet = LOS_BinarySemCreate(1, &BinarySem_Handle);
5 if (uwRet != LOS_OK)
6 {
7     printf("BinarySem 二值信号量创建失败! \n");
8 }
```

6.5.3 计数信号量创建函数 LOS_SemCreate()

计数信号量的最大容量则为 OS_SEM_COUNTING_MAX_COUNT,该宏定义的值 0xFFFF。计数信号量创建函数 LOS_SemCreate()的函数原型如代码清单 6.4 所示。

代码清单 6.4 LOS_SemCreate()函数原型

```
1 LITE_OS_SEC_TEXT_INIT UINT32 LOS_SemCreate (UINT16 usCount, UINT32
*puwSemHandle)
```

在创建信号量的时候,只需要传入用户定义的保存二值信号量 ID 的变量地址 (puwSemHandle) 与初始化可用信号量个数的值 (usCount)。

6.5.4 信号量删除函数 LOS_SemDelete()

信号量删除函数是根据信号量 ID 直接删除的,删除之后信号量的所有信息都会被系统回收,而且不能再次使用这个信号量,但是需要注意:

- (1)信号量在使用或者有任务在阻塞等待该信号量的时候是不能被删除的；
- (2) 如果某个信号量没有被创建，那也是无法被删除的。

删除函数 `LOS_SemDelete()`的函数原型如代码清单 6.5 所示。

代码清单 6.5 `LOS_SemDelete()`函数原型

```
1 LITE_OS_SEC_TEXT_INIT UINT32 LOS_SemDelete(UINT32 uwSemHandle)
```

`uwSemHandle` 是信号量 ID，表示要删除的信号量。

6.5.5 信号量释放函数 `LOS_SemPost()`

信号量的释放可以在任务、中断中使用。

在前面的讲解中，读者已经了解到只有当信号量有效的时候，任务才能获取信号量，那么，是什么函数使得信号量变得有效呢？其实有两种方式：

(1) 在创建信号量的时候指定可用的信号量个数的初始值。在二进制信号量中，该初始值的范围是 0~1，假如某个信号量中可用信号量个数为 1，那么在信号量被获取一次后就成为无效状态，这时候就需要在其他任务或中断释放信号量；

(2) 调用信号量释放函数 `LOS_SemPost()`。每调用一次该函数就释放一个信号量，可以将信号量由无效状态转为有效状态。但无论是二值信号量还是计数信号量，都不能一直释放信号量，需要注意可用信号量的范围，对于二值信号量必须确保其可用值在 0~1（`OS_SEM_BINARY_MAX_COUNT`）范围内；而计数信号量其范围是 0~`OS_SEM_COUNTING_MAX_COUNT`（`0xFFFF`）。

信号量释放函数 `LOS_SemPost()`的函数原型如代码清单 6.6 所示。

代码清单 6.6 `LOS_SemPost()`函数原型

```
1 LITE_OS_SEC_TEXT UINT32 LOS_SemPost(UINT32 uwSemHandle)
```

`uwSemHandle` 是信号量 ID，表示要释放的信号量。

因为信号量的释放是直接调用 `LOS_SemPost()`函数，是没有阻塞情况的，所以可以在中断中调用 `LOS_SemPost()`函数。信号量释放函数 `LOS_SemPost()`的应用实例如代码清单 6.7 加粗部分所示。

代码清单 6.7 `LOS_SemPost()`函数应用实例

```
1 static void Write_Task(void)
2 {
```

```

3  while(1){
4  //获取二值信号量 BinarySem_Handle, 没获取到则一直等待
5  LOS_SemPend(BinarySem_Handle , LOS_WAIT_FOREVER);
6  ucValue[0]++;
7  LOS_TaskDelay(100); /* 延时 100Ticks */
8  ucValue[1]++;
9  LOS_SemPost(BinarySem_Handle); //释放二值信号量 BinarySem_Handle
10 LOS_TaskYield(); //放弃剩余时间片, 进行一次任务切换, 并将该任务移到同等优先级就绪
11     //任务队列的尾部
12 }
13 }

```

6.5.6 信号量获取函数 LOS_SemPend()

与释放信号量对应的是获取信号量, 当信号量有效的时候, 任务才能获取信号量。任务获取了某个信号量时, 该信号量的可用个数减一, 当它为 0 的时候, 获取信号量的任务会进入阻塞态, 阻塞时间由用户指定。每调用一次 LOS_SemPend() 函数获取信号量, 信号量的可用个数减一, 直至为 0, LOS_SemPend() 的函数原型如代码清单 6.8 所示。

代码清单 6.8 LOS_SemPend() 函数原型

```

1 LITE_OS_SEC_TEXT UINT32 LOS_SemPend(UINT32 uwSemHandle, UINT32 uwTimeout)

```

uwSemHandle 是信号量 ID, 表示要获取的信号量, uwTimeout 是用户指定的等待时间。

注意: LiteOS 禁止在中断服务程序中获取信号量。

信号量获取函数 LOS_SemPend() 的应用实例如代码清单 6.9 加粗部分所示。

代码清单 6.9 信号量获取函数 LOS_SemPend() 应用实例

```

1 static void Read_Task(void)
2 {
3  while(1){
4  //获取二值信号量 BinarySem_Handle, 没获取到则一直等待
5  LOS_SemPend( BinarySem_Handle , LOS_WAIT_FOREVER );
6  if ( ucValue[0] == ucValue[1] ){
7      printf( "\r\nSuccessful\r\n" );
8  }
9  else{
10     printf( "\r\nFail\r\n" );
11 }
12 LOS_SemPost(BinarySem_Handle); //释放二值信号量 BinarySem_Handle

```

```
13     LOS_TaskDelay(1000); //每 1s 读一次，延时 1000 个 Tick
14 }
15 }
```

信号量有三种获取模式：

（1）无阻塞模式，任务获取信号量时，如果当前信号量中可用信号量个数不为 0，则获取成功，否则立即返回获取失败。

（2）永久阻塞模式，任务获取信号量时，如果当前信号量中可用信号量个数不为 0，则获取成功，否则该任务进入阻塞态，直到有其他任务/中断释放该信号量。

（3）指定阻塞时间模式，任务获取信号量时，如果当前信号量中可用信号量个数不为 0，则获取成功，否则该任务进入阻塞态，阻塞时间由用户指定，在这段时间中有其他任务/中断释放该信号量，任务将恢复就绪态；或当阻塞时间超时，任务也会恢复就绪态。

第 7 章 互斥锁

在学习信号量的时候，曾提到过“互斥”，但是在操作系统中使用信号量做临界资源的互斥保护并不是最明智的选择，LiteOS 提供另一种机制——互斥锁，专门用于临界资源互斥保护。

7.1 互斥锁的基本概念

互斥锁又称互斥信号量，是一种特殊的二值信号量，它和信号量不同的是，它具有互斥锁所有权、递归访问以及优先级继承等特性，常用于实现对临界资源的独占式处理。任意时刻互斥锁的状态只有两种，开锁或闭锁。当互斥锁被任务持有时，该互斥锁处于闭锁状态，任务获得互斥锁的所有权。当该任务释放互斥锁时，该互斥锁处于开锁状态，任务失去该互斥锁的所有权。**当一个任务持有互斥锁时，其他任务将不能再对该互斥锁进行开锁或持有。**持有该互斥锁的任务能够再次获得这个锁而不被挂起，这就是互斥量的递归访问，这个特性与一般的信号量有很大的不同，在信号量中，由于已经不存在可用的信号量，任务递归获取信号量时会发生主动挂起任务，最终形成死锁。

如果想要实现同步功能（任务与任务间或者任务与中断间同步），二值信号量或许是更好的选择，虽然互斥锁也可以用于任务与任务间的同步，但互斥锁更多的是用于保护资源的互斥访问。

互斥锁可以充当保护资源的令牌，当一个任务希望访问某个资源时，它必须先获取令牌；当任务使用资源后，必须归还令牌，以便其他任务可以访问该资源。

在二值信号量中也可以用于保护临界资源，当任务获取信号量后才能开始使用该资源，当不需要使用时就释放信号量，如此一来其他任务也能获取到信号量从而可用使用资源。但**信号量会导致的另一个潜在问题：可能发生任务优先级翻转。**而 LiteOS 提供的互斥锁可以通过优先级继承算法，降低优先级翻转产生的危害，**所以在临界资源的保护中一般使用互斥锁。**

7.2 互斥锁的优先级继承机制

在 LiteOS 中为了降低优先级翻转产生的危害使用了优先级继承算法，优先级继承算法是指：暂时提高占有某种临界资源的低优先级任务的优先级，使它的优先级与等待该资源的最高优先级任务相同。当这个低优先级任务执行完毕释放该资源时，优先级恢复初始设定值。这个过程可以看作是低优先级任务继承了高优先级任务的优先级。因此，继承高优先级的低优先级任务避免了系统资源被任何中间优先级的任务抢占，从而降低对高优先级任务响应时间的影响。

互斥锁与二值信号量最大的区别是：互斥锁具有优先级继承机制，而信号量没有。也就是说，若某个临界资源受到一个互斥锁保护。任务访问该资源时需要获得互斥锁，如果这个资源正在被一个低优先级任务使用，那么此时的互斥锁是闭锁状态，其他任务不能获得该互斥锁，如果此时一个高优先级任务想要访问该资源，那么高优先级任务会因为获取不到互斥锁而进入阻塞态，系统会将当前持有该互斥锁的任务的优先级临时提升到与高优先级任务相同，这就是**优先级继承机制**。它确保高优先级任务进入阻塞状态的时间尽可能短，以及将已经出现的“优先级翻转”危害降低到最小。

任务的优先级在任务创建的时候就已经指定，高优先级的任务可以打断低优先级的任务，抢占 CPU 的使用权。但是在很多场合中，某些资源只有一个，**当低优先级任务正在占用该资源的时候，互斥锁处于闭锁状态，即便是高优先级任务也只能等待低优先级任务释放互斥锁**，然后高优先级任务才能获得互斥锁去访问该资源。高优先级任务无法运行而低优先级任务可以运行的现象称为“优先级翻转”。

为什么说优先级翻转在操作系统中危害很大？因为在开始设计系统的时候，就已经根据重要程度指定了任务的优先级，越重要的任务优先级越高。但是发生优先级翻转，会导致系统的高优先级任务阻塞时间过长，得不到及时处理，有可能对整个系统产生严重的危害，同时也违反了操作系统可抢占调度的原则。

举个例子，当前系统中存在三个任务，分别为 H 任务（High）、M 任务（Middle）、L 任务（Low），三个任务的优先级顺序为“H 任务>M 任务>L 任务”。正常运行时 H 任务可以打断 M 任务与 L 任务，M 任务可以打断 L 任务。假设系统中存在一个临界资源，此时该资源被 L 任务正在使用中，某一时刻，H 任务需要使用该资源，但此时 L 任务还未释放资源，H 任务则因为

获取不到该资源使用权而进入阻塞态，L 任务继续使用该资源，此时已经出现了“优先级翻转”现象，高优先级任务在等待低优先级的任务执行，如果在 L 任务执行的时候刚好 M 任务被唤醒了，由于 M 任务优先级比 L 任务优先级高，那么会打断 L 任务，抢占 L 任务的 CPU 使用权，直到 M 任务执行完，再把 CPU 使用权归还给 L 任务，L 任务继续执行，等到执行完毕之后释放该资源，此时 H 任务才能从阻塞态解除，使用该资源。这个过程中，本来是最高优先级的 H 任务，等待了更低优先级的 L 任务与 M 任务，其阻塞的时间是 M 任务运行时间+L 任务运行时间，假如系统中有多多个 M 任务这样的中间优先级任务抢占最低优先级任务 CPU 使用权，那么系统最高优先级的任务将持续阻塞，这是绝对不允许出现的。因此在没有优先级继承的情况下，保护临界资源将有可能产生优先级翻转，其危害极大，优先级翻转过程示意图如图 7.1 所示。

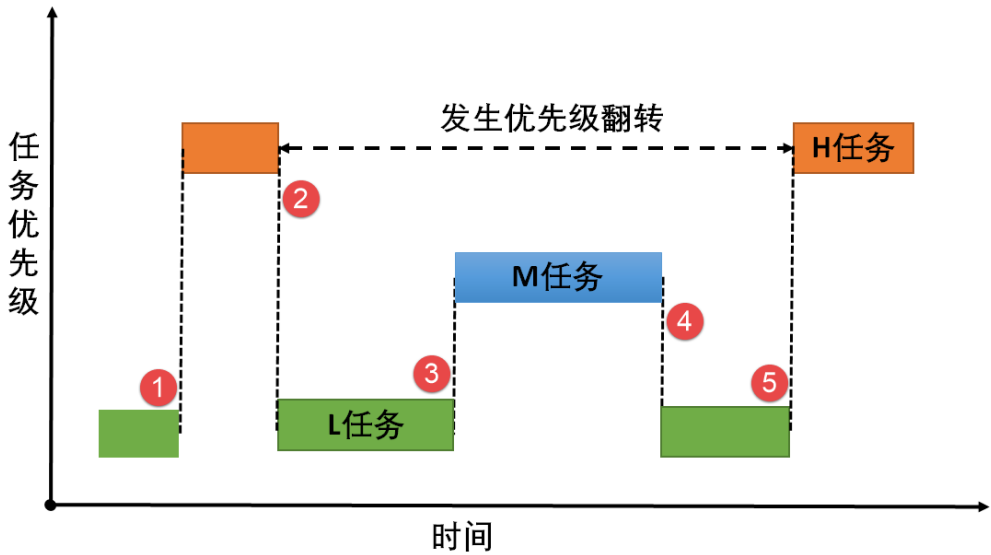


图 7.1 优先级翻转示意图

图 7.1(1): L 任务正在使用某临界资源，H 任务被唤醒，执行 H 任务。但 L 任务并未执行完毕，此时临界资源还未释放。

图 7.1(2): 这个时刻 H 任务也要对该临界资源进行访问，但 L 任务还未释放资源，由于保护机制，H 任务进入阻塞态，L 任务得以继续运行，此时已经发生了优先级翻转现象。

图 7.1(3): 某个时刻 M 任务被唤醒，由于 M 任务的优先级高于 L 任务，M 任务抢占了 CPU 的使用权，M 任务开始运行，此时 L 任务尚未执行完，临界资源还未被释放。

图 7.1(4): M 任务运行结束，归还 CPU 使用权，L 任务继续运行。

图 7.1(5): L 任务运行结束, 释放临界资源, H 任务得以对资源进行访问, H 任务开始运行。

如果 H 任务的等待时间过长, 对整个系统来说可能是致命的, 所以尽可能降低高优先级任务的等待时间以降低优先级翻转的危害。而互斥锁由于其特有的优先级继承机制可以降低优先级翻转产生的危害。

假如系统使用互斥锁保护临界资源, 那么就具有优先级继承特性, 任务需要在获取互斥锁后才能访问临界资源。在 H 任务获取互斥锁时, 由于获取不到互斥锁而进入阻塞态, 那么系统就会把当前正在使用资源的 L 任务的优先级临时提升到与 H 任务优先级相同, 当 M 任务被唤醒时, 因为它的优先级比 H 任务低, 所以无法打断 L 任务, 因为此时 L 任务的优先级被临时提升到与 H 任务相同, 所以当 L 任务使用完该资源释放互斥锁, H 任务将获得互斥锁而恢复运行。因此 H 任务的阻塞时间仅仅是 L 任务的执行时间, 此时优先级翻转的危害降到了最低, 这就是优先级继承的优势, 其示意图如图 7.2 所示。

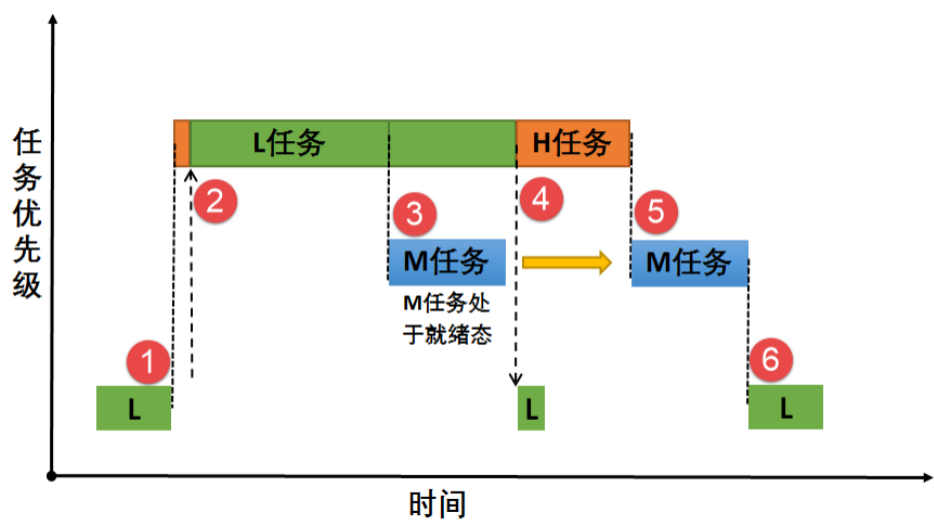


图 7.2 优先级继承示意图

图 7.2(1): L 任务正在使用某临界资源, H 任务被唤醒, 执行 H 任务。但 L 任务尚未运行完毕, 此时互斥锁还未释放。

图 7.2(2): 某一时刻 H 任务也要获取互斥锁访问该资源, 由于互斥锁对临界资源的保护机制, H 任务无法获得互斥锁而进入阻塞态。此时发生优先级继承, 系统将 L 任务的优先级暂时提升到与 H 任务优先级相同, L 任务继续执行。

图 7.2(3): 在某一时刻 M 任务被唤醒, 由于此时 M 任务的优先级暂时低于 L 任务, 所以 M 任务仅在就绪态, 而无法获得 CPU 使用权。

图 7.2(4): L 任务运行完毕释放互斥锁, H 任务获得互斥锁后恢复运行, 此时 L 任务的优先级会恢复初始指定的优先级。

图 7.2(5): 当 H 任务运行完毕, M 任务得到 CPU 使用权, 开始执行。

图 7.2(6): 系统正常运行, 按照初始指定的优先级运行。

使用互斥锁的时候一定需要注意以下几点:

- (1) 在获得互斥锁后, 须尽快释放互斥锁。
- (2) 在任务持有互斥锁的这段时间, 不得更改任务的优先级。
- (3) LiteOS 的优先级继承机制不能解决优先级翻转, 只能将这种情况的影响降低到最小, 硬实时系统在一开始设计时就要避免优先级翻转发生。
- (4) 互斥锁不能在中断服务程序中使用。

7.3 互斥锁的应用场景

互斥锁的使用比较单一, 因为它是信号量的一种, 并且它是以锁的形式存在。在初始化的时候, 互斥锁处于开锁的状态, 而当被任务持有的时候则立刻转为闭锁的状态。互斥锁更适合于以下场景。

- (1) 可能会引起优先级翻转的情况。
- (2) 任务可能会多次获取互斥锁的情况下。这样可以避免同一任务多次递归持有而造成死锁的问题。

多任务环境下往往存在多个任务竞争同一临界资源的应用场景, 互斥锁可被用于对临界资源的保护从而实现独占式访问。另外, 互斥锁可以降低信号量存在的优先级翻转问题带来的影响。

比如有两个任务需要使用串口进行数据发送, 串口资源只有一个, 那么两个任务肯定不能同时发送数据, 否则将导致数据错误, 那么就可以用互斥锁对串口资源进行保护。当一个任务使用串口的时候, 先获取互斥锁, 互斥锁立即变为闭锁状态, 另一个任务因获取不到互斥锁而不能访问串口资源。直到任务使用完串口释放互斥锁后, 另一个任务才能获取互斥锁从而得到串口资源的使用权。

7.4 互斥锁的使用

7.4.1 互斥锁控制块

互斥锁控制块与信号量控制块类似，系统中每一个互斥锁都有对应的互斥锁控制块，它记录了互斥锁的所有信息，比如互斥锁的状态，持有次数、ID、所属任务等，如代码清单 7.1 所示。

代码清单 7.1 互斥锁控制块

```
1 typedef struct {
2     UINT8 ucMuxStat;           (1)
3     UINT16 usMuxCount;        (2)
4     UINT32 ucMuxID;           (3)
5     LOS_DL_LIST stMuxList;    (4)
6     LOS_TASK_CB *pstOwner;    (5)
7     UINT16 usPriority;         (6)
8 } MUX_CB_S;
```

代码清单 7.1(1)：ucMuxStat 是互斥锁状态，其状态有两个，OS_MUX_UNUSED 或 OS_MUX_USED，表示互斥锁是否被使用。

代码清单 7.1(2)：usMuxCount 是互斥锁持有次数，在每次获取互斥锁的时候，该成员变量会增加，用于记录持有的次数，当 usMuxCount 为 0 的时候表示互斥锁处于开锁状态，任务可以随时获取，当它是一个正值的时候，表示互斥锁已经被获取了，只有持有互斥锁的任务才能释放它。

代码清单 7.1(3)：ucMuxID 是互斥锁 ID。

代码清单 7.1(4)：stMuxList 是互斥锁阻塞列表。用于记录阻塞在此互斥锁的任务。

代码清单 7.1(5)：*pstOwner 是一个任务控制块指针，指向当前持有该互斥锁任务，这样系统就能够知道该互斥锁的所有权归哪个任务，互斥锁只能由持有互斥锁的任务释放，其他任务都没有权利操作已经处于闭锁状态的互斥锁。

代码清单 7.1(6)：usPriority 是记录持有互斥锁任务的初始优先级，用于处理优先级继承。

7.4.2 互斥锁创建函数 LOS_MuxCreate()

LiteOS 提供互斥锁创建函数接口——LOS_MuxCreate()，该函数用于创建一个互斥锁，在创建互斥锁后，系统会返回互斥锁 ID，以后通过互斥锁 ID 对互斥锁进行操作，因此需要用户定义一个互斥锁 ID 变量，并将变量的地址传入互斥锁创建函数中，LOS_MuxCreate()函数原型如代码清单 7.2 所示，其应用实例如代码清单 7.3 加粗部分所示。

代码清单 7.2 LOS_MuxCreate()函数原型

```
1 LITE_OS_SEC_TEXT_INIT UINT32 LOS_MuxCreate (UINT32 *puwMuxHandle)
```

代码清单 7.3 互斥锁创建函数 LOS_MuxCreate()应用实例

```
1 /* 定义互斥锁的 ID 变量 */
2 UINT32 Mutex_Handle;
3 UINT32 uwRet = LOS_OK; /* 定义一个创建任务的返回类型，初始化为创建成功的返回值 */
4
5 /* 创建一个互斥锁*/
6 uwRet = LOS_MuxCreate(&Mutex_Handle);
7 if(uwRet != LOS_OK)
8 {
9     printf("Mutex_Handle 互斥锁创建失败! \n");
10 }
```

7.4.3 互斥锁删除函数 LOS_MuxDelete()

用户可以根据互斥锁 ID 将互斥锁删除，删除后的互斥锁将不能被使用，它所有信息都会被系统回收，如果系统中有任务持有互斥锁或者有任务阻塞在互斥锁时，互斥锁是不能被删除的。uwMuxHandle 是互斥锁 ID，表示要删除的互斥锁，其函数原型如代码清单 7.4 所示。

代码清单 7.4 LOS_MuxDelete()函数原型

```
1 LITE_OS_SEC_TEXT_INIT UINT32 LOS_MuxDelete(UINT32 uwMuxHandle)
```

互斥锁删除函数的使用方法，如代码清单 7.5 加粗部分所示。

代码清单 7-5 互斥锁删除函数 LOS_MuxDelete()实例

```
1 UINT32 uwRet = LOS_OK; /* 定义一个返回类型，初始化为删除成功的返回值 */
2 uwRet = LOS_MuxDelete(Mutex_Handle); /* 删除互斥锁 */
3 if (LOS_OK == uwRet)
```

```
4 {
5     printf("互斥锁删除成功! \n");
6 }
```

7.4.4 互斥锁释放函数 LOS_MuxPost()

任务想要访问某个临界资源时,需要先获取互斥锁,然后才能访问该资源,在任务使用完该资源后必须要及时释放互斥锁,其他任务才能获取互斥锁从而访问该资源。在前面章节的讲解中,读者应该都知道当互斥锁处于开锁状态的时候,任务才能获取互斥锁,那么,是什么函数使互斥锁处于开锁状态呢? LiteOS 提供了互斥锁释放函数 LOS_MuxPost(),持有互斥锁的任务可以调用该函数将互斥锁释放,释放后的互斥锁处于开锁状态,系统中其他任务可以获取互斥锁。但互斥锁允许在任务中释放而不能在中断中释放,原因有以下两点:

- (1) 中断上下文没有一个任务的概念。
- (2) 互斥锁只能被持有者释放,持有者是任务。

互斥锁有所属关系,只有持有者才能释放互斥锁,而这个持有者是任务,因为中断上下文没有任务概念,所以中断上下文不能持有,也不能释放互斥锁。

使用该函数接口时,只有已持有互斥锁所有权的任务才能释放它,当持有互斥锁的任务调用 LOS_MuxPost()函数时会将互斥锁变为开锁状态,如果有其他任务在等待获取该互斥锁时,等待的任务将被唤醒,然后持有该互斥锁。如果任务的优先级被临时提升,那么当互斥锁被释放后,任务的优先级将恢复为初始设定的优先级,LOS_MuxPost()函数原型如代码清单 7.6 所示。

代码清单 7.6 LOS_MuxPost()函数原型

```
1 LITE_OS_SEC_TEXT UINT32 LOS_MuxPost(UINT32 uwMuxHandle)
```

互斥锁释放函数的应用实例如代码清单 7.7 加粗部分所示。

代码清单 7.7 互斥锁释放函数 LOS_MuxPost()应用实例

```
1 /* 定义互斥锁的 ID 变量 */
2 UINT32 Mutex_Handle;
3
4 UINT32 uwRet = LOS_OK; /* 定义一个返回类型,初始化为成功的返回值 */
5 /* 释放一个互斥锁 */
6 uwRet = LOS_MuxPost(Mutex_Handle);
7 if (LOS_OK == uwRet)
8 {
```

```
9     printf("互斥锁释放成功! \n");
10 }
```

7.4.5 互斥锁获取函数 LOS_MuxPend()

当互斥锁处于开锁状态时，任务才能够获取互斥锁，当任务持有了某个互斥锁的时候，其他任务就无法获取这个互斥锁，需要等到持有互斥锁的任务释放后，其他任务才能获取成功，任务通过互斥锁获取函数来获取互斥锁的所有权。任务对互斥锁的所有权是独占的，任意时刻互斥锁只能被一个任务持有，如果互斥锁处于开锁状态，那么获取该互斥锁的任务将成功获得并拥有互斥锁的使用权；如果互斥锁处于闭锁状态，获取该互斥锁的任务将无法获得互斥锁，任务将被挂起，在任务被挂起之前，会进行优先级继承，如果当前任务优先级比持有互斥锁的任务优先级高，那么将会临时提升持有互斥锁任务的优先级。互斥锁的获取函数是 LOS_MuxPend()，其函数原型如代码清单 7.8 所示。

代码清单 7.8 LOS_MuxPend()函数原型

```
9 LITE_OS_SEC_TEXT UINT32 LOS_MuxPend(UINT32 uwMuxHandle, UINT32 uwTimeout)
```

需要注意互斥锁是不允许在中断服务程序中操作，互斥锁获取函数的应用实例如代码清单 7.9 加粗部分所示。

代码清单 7.9 互斥锁获取函数 LOS_MuxPend()应用实例

```
1 /* 定义互斥锁的 ID 变量 */
2 UINT32 Mutex_Handle;
3
4 UINT32 uwRet = LOS_OK; /* 定义一个返回类型，初始化为成功的返回值 */
5 //获取互斥锁，没获取到则一直等待
6 uwRet = LOS_MuxPend(Mutex_Handle, LOS_WAIT_FOREVER);
7 if (LOS_OK == uwRet)
8 {
9     printf("互斥获取成功! \n");
10 }
```

7.4.6 使用互斥锁的注意事项

(1) 两个任务不能获取同一个互斥锁。如果某任务尝试获取已被持有的互斥锁，则该任务会被阻塞，直到持有该互斥锁的任务释放互斥锁。

(2) 互斥锁不能在中断服务程序中使用。

(3) LiteOS 作为实时操作系统需要保证任务调度的实时性，尽量避免任务的长时间阻塞，因此在获得互斥锁之后，应该尽快释放互斥锁。

(4) 任务持有互斥锁的过程中，不允许再调用 `LOS_TaskPriSet()` 等函数接口更改持有互斥锁任务的优先级。

(5) 互斥锁和信号量的区别在于：互斥锁可以被已经持有互斥锁的任务重复获取，而不会形成死锁。这个递归调用功能是通过互斥锁控制块 `usMuxCount` 成员变量实现的，这个变量用于记录任务持有互斥锁的次数，在每次获取互斥锁后该变量加 1，在释放互斥锁后该变量减 1。只有当 `usMuxCount` 的值为 0 时，互斥锁才处于开锁状态，其他任务才能获取该互斥锁。

第 8 章 事件

8.1 事件的基本概念

事件是一种实现任务间通信的机制，主要用于实现多任务间的同步，**只能实现事件类型的通信**，而**无数据传输**。事件与信号量不同，它**可以实现一对多，多对多**的同步处理。

即**一个任务可以等待多个事件的发生**，可以是任意一个事件发生时唤醒任务，也可以是几个事件都发生后才唤醒任务。同样，系统也允许多个事件同步多个任务。

每一个事件控制块只需要很少的内存空间来保存事件信息，事件标记存储在一个 UIN32 类型的变量 `uwEventID` 中，该变量在事件控制块中定义，每一位代表一个事件，可以称之为事件集合，任务通过“逻辑与”或“逻辑或”与一个或多个事件建立关联，可以称该变量为事件集合。事件的“逻辑或”也被称作是独立型同步，在任务感兴趣的若干个事件中，任意一个事件发生时任务均可被唤醒；事件“逻辑与”则被称为是关联型同步，在任务感兴趣的若干个事件都发生时才会被唤醒，事件发生的时间可以不同步。

在多任务环境下，任务与任务、任务与中断之间往往需要同步操作，一个事件的发生会告知等待中的任务，即形成一个任务与任务、中断与任务间的同步。LiteOS 事件可以提供一对多、多对多的同步操作。

一对多同步模型：一个任务等待多个事件的触发；

多对多同步模型：多个任务等待多个事件的触发。

任务可以通过设置事件位来实现事件的触发和等待操作。LiteOS 提供的事件具有如下特点：

(1) 事件相互独立，一个 32 位的变量（事件集合），用于标识该任务发生的事件类型，其中每一位表示一种事件类型（0 表示该事件类型未发生、1 表示该事件类型已经发生），一共 31 种事件类型（第 25 位保留）。

(2) 事件不提供传输数据功能。

(3) 事件无排队性，即多次向任务设置同一事件（如果任务还未来得及读

走），等效于只设置一次。

（4）允许多个任务对同一事件进行读写操作。

（5）支持事件等待超时机制。

在 LiteOS 中，每个事件获取的时候，可以指定任务感兴趣的事件，并且选择读取事件信息标记，它有三个属性，分别是逻辑与（`LOS_WAITMODE_AND`），逻辑或（`LOS_WAITMODE_OR`）以及是否清除事件标记（`LOS_WAITMODE_CLR`）。当任务等待事件同步时，可以通过任务感兴趣的事件位和事件信息标记来判断当前接收的事件是否满足要求，如果满足则说明任务等待到对应的事件，系统将唤醒等待的任务；否则，任务会根据用户指定的阻塞时间继续等待下去。

8.2 事件的应用场景

LiteOS 的事件用于事件类型的通信，无数据传输，即可以用事件做标志位，判断某些事件是否发生了，然后根据结果做处理。可能读者有疑问，为什么不直接用变量做标志呢？其实若是在裸机编程中，用全局变量是最为有效的方法，但在操作系统中，使用全局变量就要考虑以下问题：

（1）如何对全局变量进行保护？如何处理多任务同时对它进行访问？

（2）如何让内核对事件进行有效管理？使用全局变量的话，就需要在任务中轮询查看事件是否发送，这会浪费大量的 CPU 资源。除此之外还有阻塞超时机制。

所以，在操作系统中，还是使用操作系统提供的通信机制更为简单方便，LiteOS 事件具有以下优点：

（1）允许多个任务对同一事件进行读写操作。

（2）可以有效的解决中断服务程序和任务之间的同步问题。

（3）支持事件等待超时机制。

（4）事件发生可以立即唤醒等待中的任务。

在某些场合，可能需要多个事件发生了才能进行下一步操作，比如大型危险机器的启动，需要检查各项指标，当指标不达标的时候，无法启动，所以需要事件做等待处理，当所有的指标都检测完毕，机器才允许启动。

事件可使用于多种场合，它能够在一定程度上替代信号量，用于任务与任

务间，中断与任务间的同步。一个任务或中断服务程序可以写入事件，那么等待对应事件的任务将被唤醒并进行处理。但是它与信号量不同的是，事件的写操作是不可累计的，而信号量的释放动作是可累计的。事件另外一个特性是，任务可等待多个事件发生。此外任务还可以按照需求选择是“逻辑或”，还是“逻辑与”读取事件。而信号量只能识别单一同步动作，不能同时等待多个信号的同步。

各个事件可分别发送或一起写入事件集合中，任务仅对需要的事件进行关注即可，当事件发生时并且满足任务唤醒的条件，任务将被唤醒并执行后续的处理动作。

8.3 事件的运作机制

任务可以根据事件类型(事件掩码)uwEventMask 读取单个或者多个事件，事件读取成功后，如果读取模式设置为 LOS_WAITMODE_CLR 则会清除已读取到的事件类型，反之不会清除已读到的事件类型。用户可以选择读取事件的模式，读取事件类型中的所有事件或者是任意事件。

任务/中断可以写入指定的事件类型（事件掩码），设置事件集合的某些位为 1，系统支持写入多个事件类型，写事件成功可能会触发任务调度。

清除事件时，根据事件控制块和待清除的事件类型，对事件对应位进行清 0 操作。

事件控制块中有一个 32 位的变量 uwEventID，可以称之为事件集合，它用于标识该任务发生的事件类型，其中每一位表示一种事件类型（0 表示该事件类型未发生、1 表示该事件类型已经发生），一共 31 种事件类型（第 25 位保留），如图 8.1 所示。

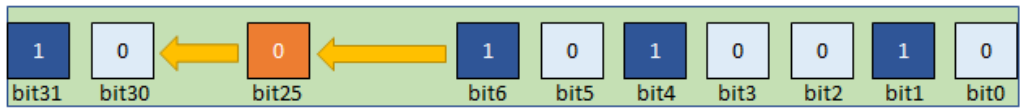


图 8.1 事件集合（一个 32 位的变量）

事件唤醒机制，当任务因为等待某个或者多个事件发生而进入阻塞态，当事件发生的时候会被唤醒，其过程如图 8.2 所示。

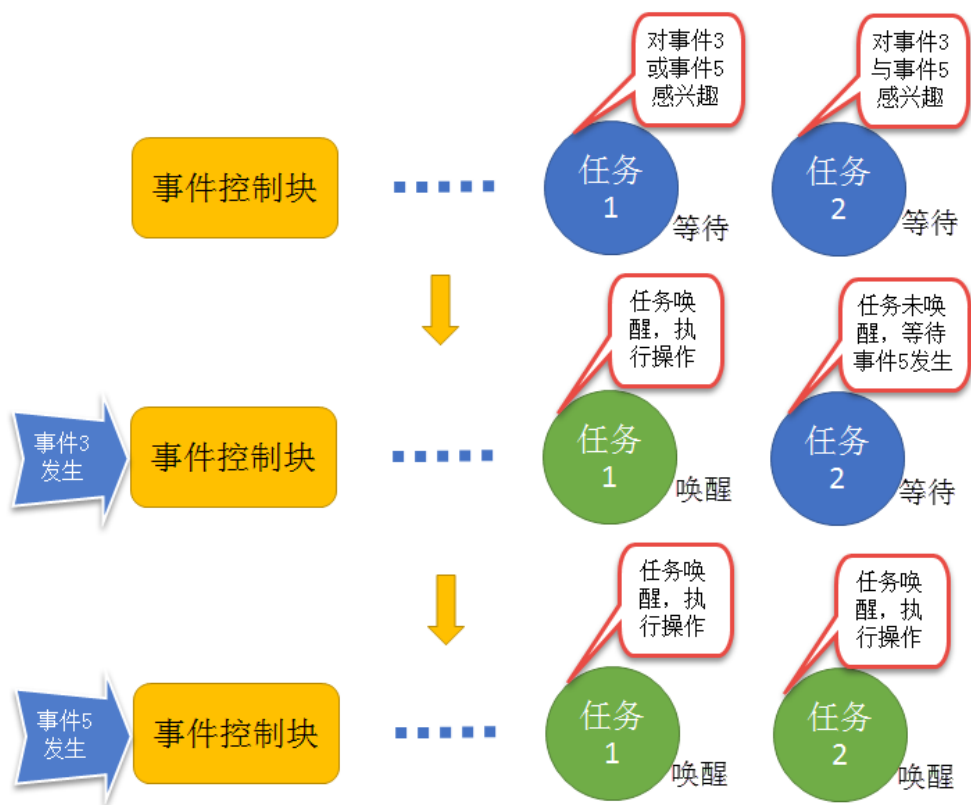


图 8.2 事件唤醒任务示意图

任务 1 对事件 3 或事件 5 感兴趣（逻辑或 `LOS_WAITMODE_OR`），当发生其中的某一个事件都会被唤醒，并且执行相应操作。而任务 2 对事件 3 与事件 5 感兴趣（逻辑与 `LOS_WAITMODE_AND`），当且仅当事件 3 与事件 5 都发生的时候，任务 2 才会被唤醒，如果只有其中一个事件发生，那么任务 2 还是会继续等待事件发生。如果在读事件函数中设置了清除事件位 `LOS_WAITMODE_CLR`，那么当任务 2 被唤醒后，系统会把事件 3 和事件 5 的事件位清零。

8.4 事件使用讲解

8.4.1 事件控制块

系统都是通过事件控制块对事件进行操作的，事件控制块中包含了一个 32 位的 `uwEventID` 变量，其变量的各个位表示一个事件，此外还存在一个事件链表 `stEventList`，用于记录等待事件的任务，所有在等待此事件的任务，事件控

制块结构如代码清单 8.1 所示。

代码清单 8.1 事件控制块

```
1 typedef struct tagEvent {
2     UINT32 uwEventID; /**< 事件控制块中的事件集合，指示逻辑处理的事件*/
3
4     LOS_DL_LIST stEventList; /**<事件阻塞列表*/
5 } EVENT_CB_S, *PEVENT_CB_S;
```

8.4.2 事件初始化函数 LOS_EventInit()

LiteOS 提供事件的初始化函数——LOS_EventInit()，它需要用户定义一个事件控制块结构体变量，然后将事件控制块的变量地址通过 pstEventCB 参数传递到事件初始化函数中，LOS_EventInit()函数原型如代码清单 8.2 所示，应用实例如代码清单 8.3 加粗部分所示。

代码清单 8.2 LOS_EventInit()函数原型

```
1 LITE_OS_SEC_TEXT_INIT UINT32 LOS_EventInit(PEVENT_CB_S pstEventCB)
```

代码清单 8.3 事件初始化函数 LOS_EventInit()应用实例

```
1 /* 定义事件标志组的控制块 */
2 static EVENT_CB_S EventGroup_CB;
3 UINT32 uwRet = LOS_OK; /* 定义一个返回类型，初始化为成功的返回值 */
4 /* 初始化一个事件标志组*/
5 uwRet = LOS_EventInit(&EventGroup_CB);
6 if (uwRet != LOS_OK)
7 {
8     printf("EventGroup_CB 事件标志组初始化失败! \n");
9 }
```

8.4.3 事件销毁函数 LOS_EventDestory()

在某些场合中事件可能只需要使用一次，如危险机器的启动，假如各项指标都达到了，并且机器启动成功了，那事件可能就不会重复使用，那么可以进行销毁事件。LiteOS 提供了一个销毁事件的函数——LOS_EventDestory()，其函数原型如代码清单 8.4 所示，应用实例如代码清单 8.5 加粗部分所示。

代码清单 8.4 LOS_EventDestory()函数原型

```
1 LITE_OS_SEC_TEXT_INIT UINT32 LOS_EventDestory(PEVENT_CB_S pstEventCB)
```

```

1  /* 定义事件标志组的控制块 */
2  static EVENT_CB_S EventGroup_CB;
3  UINT32 uwRet = LOS_OK; /* 定义一个返回类型，初始化为成功的返回值 */
4  /* 销毁一个事件标志组 */
5  uwRet = LOS_EventDestory(&EventGroup_CB);
6  if(uwRet != LOS_OK)
7  {
8      printf("EventGroup_CB 事件销毁失败! \n");
9  }

```

8.4.4 写指定事件函数 LOS_EventWrite()

此函数用于写入事件中指定的位，当位被置位之后，阻塞在该位上的任务将会被解锁。使用该函数接口时，通过指定事件设置对应的标志位，然后遍历阻塞在事件列表上的任务，判断是否满足任务唤醒条件，如果满足则唤醒该任务。需要注意的是 uwEventID 的第 25 位是 LiteOS 保留出来的，原因是为了区别读事件函数 LOS_EventRead() 返回的是事件还是错误代码，LOS_EventWrite() 函数原型如代码清单 8.6 所示。

代码清单 8.6 LOS_EventWrite()函数原型

```

1  LITE_OS_SEC_TEXT UINT32 LOS_EventWrite(PEVENT_CB_S pstEventCB, UINT32
uwEvents)

```

如果想要记录一个事件的发生，这个事件在事件集合的位置是 bit0，当事件还未发生时，事件集合 bit0 为 0，**当它发生时，用户只需要往事件集合 bit0 中写入 1，就表示事件已经发生了。**为了便于理解，一般操作都是用宏定义来实现，如 #define EVENT (0x01 << x)，“<< x”表示写入事件集合的 bit x，如代码清单 8.7 加粗部分所示。

代码清单 8.7 写指定事件函数 LOS_EventWrite()应用实例

```

1  /* 定义事件标志组的控制块 */
2  static EVENT_CB_S EventGroup_CB;
3
4  #define KEY1_EVENT (0x01 << 0) //设置事件掩码的位 0
5  #define KEY2_EVENT (0x01 << 1) //设置事件掩码的位 1
6
7  static void Key_Task(void)
8  {

```

```

9  while (1){//如果 KEY1 被按下
10     if( Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN) == KEY_ON ) {
11         //LED1_ON; //点亮 LED1
12         printf ( "KEY1 被按下\n");
13         //置位事件标志组的 BIT0
14         LOS_EventWrite(&EventGroup_CB, KEY1_EVENT);
15     }//如果 KEY2 被按下
16     if( Key_Scan(KEY2_GPIO_PORT,KEY2_GPIO_PIN) == KEY_ON) {
17         //LED2_ON; //点亮 LED2
18         printf ( "KEY2 被按下\n");
19         LOS_EventWrite(&EventGroup_CB, KEY2_EVENT); //置位事件标志组的 BIT1
20     }
21     LOS_TaskDelay(20);
22 }
23 }

```

8.4.5 读指定事件函数 LOS_EventRead()

LiteOS 提供了一个读取指定事件的函数——LOS_EventRead(), 通过这个函数, 就可以知道事件集合中的哪一位, 哪一个事件发生了, 然后可以通过“逻辑与”、“逻辑或”等操作对感兴趣的事件进行读取。且仅当任务等待的事件发生时, 任务才能读取到事件信息。在这段时间中, 如果事件一直没发生, 该任务将保持阻塞状态以等待事件发生。当其他任务或中断服务程序设置其等待事件对应的标志位, 并且满足读取事件的条件时, 该任务将自动由阻塞态转为就绪态。当任务阻塞时间超时, 即使事件还未发生, 任务也会自动恢复为就绪态。如果正确读取事件则返回事件集合变量的值, 由用户判断再做处理, 因为在读取事件时可能会返回不确定的值, 如果阻塞时间超时将返回错误代码, 所以需要判断任务所等待的事件是否真的发生。LOS_EventRead()函数原型如代码清单 8.8 所示。

代码清单 8.8 LOS_EventRead()函数原型

```

1 LITE_OS_SEC_TEXT UINT32 LOS_EventRead(PEVENT_CB_S pstEventCB,
2                                     UINT32 uwEventMask,
3                                     UINT32 uwMode,
4                                     UINT32 uwTimeOut)

```

当用户调用这个函数接口时, 系统首先根据用户指定参数和读取模式来判断任务要等待的事件是否发生, 如果已经发生, 则根据参数 uwMode 来决定是

否清除事件的相应标志位，并且返回事件的值，但是这个值并不是一个稳定的值，所以在等待到对应事件的时候，还需判断事件是否与任务需要的一致；如果事件没有发生，则把任务添加到事件阻塞列表中，把任务等待的事件标志值和等待模式记录下来，直到事件发生或等待时间超时，事件等待函数 `LOS_EventRead()`应用实例如代码清单 8.9 加粗部分所示。

代码清单 8.9 读指定事件函数 `LOS_EventRead()`应用实例

```
1 /* 定义事件标志组的控制块 */
2 static EVENT_CB_S EventGroup_CB;
3
4 #define KEY1_EVENT (0x01 << 0) //设置事件掩码的位 0
5 #define KEY2_EVENT (0x01 << 1) //设置事件掩码的位 1
6 /*****
7 * @ 函数名 : LED_Task
8 * @ 功能说明: 等待事件成立
9 * @ 参数 :
10 * @ 返回值 : 无
11 *****/
12 static void LED_Task(void)
13 {
14     UINT32 uwEvent;
15     while(1) {
16 /* 等待事件标志组 等待两位均被置位，读取后清除*/
17         uwEvent = LOS_EventRead(&EventGroup_CB, //事件标志组对象
18                                KEY1_EVENT|KEY2_EVENT, //等待 BIT0 和 BIT1
19                                LOS_WAITMODE_AND|LOS_WAITMODE_CLR,
20                                LOS_WAIT_FOREVER ); //无期限等待
21         if((KEY1_EVENT|KEY2_EVENT) == uwEvent){
22             printf ( "KEY1 与 KEY2 都按下\n");
23             LED1_TOGGLE; //LED1 翻转
24             //LOS_EventClear(&EventGroup_CB, ~KEY1_EVENT); //清除事件标志
25             //LOS_EventClear(&EventGroup_CB, ~KEY2_EVENT); //清除事件标志
26         }
27         else{
28             printf ( "事件错误! \n");
29         }
30     }
31 }
```

在读事件时，可以选择读取模式，读取模式如下：

(1) 所有事件 (`LOS_WAITMODE_AND`)：读取掩码中所有事件类型，

只有读取的所有事件类型都发生了，才能读取成功。

(2) 任一事件 (LOS_WAITMODE_OR)：读取掩码中任一事件类型，读取的事件中任意一种事件类型发生了，就可以读取成功。

(3) 清除事件 (LOS_WAITMODE_CLR)：LOS_WAITMODE_AND|LOS_WAITMODE_CLR 或 LOS_WAITMODE_OR|LOS_WAITMODE_CLR 时表示读取成功后，对应事件类型位会自动清除。如果模式没有设置为自动清除的话，那么需要手动显式清除。

8.4.6 清除指定事件函数 LOS_EventClear()

如果在获取事件的时候没有将对应的标志位清除，那就需要使用 LOS_EventClear() 函数显式清除事件标志，其函数原型如代码清单 8.10 所示，应用实例如代码清单 8.11 加粗部分所示。

代码清单 8.10 LOS_EventClear() 函数原型

```
1 LITE_OS_SEC_TEXT_MINOR UINT32 LOS_EventClear(PEVENT_CB_S pstEventCB, UINT32 uwEvents)
```

代码清单 8.11 清除指定事件函数 LOS_EventClear() 应用实例

```
1 /* 定义事件标志组的控制块 */
2 static EVENT_CB_S EventGroup_CB;
3
4 #define KEY1_EVENT (0x01 << 0) //设置事件掩码的位 0
5 #define KEY2_EVENT (0x01 << 1) //设置事件掩码的位 1
6 /*****
7 static void LED_Task(void)
8 {
9     UINT32 uwEvent;
10    while (1) {
11    /* 等待事件标志组 等待两位均被置位，读取后清除*/
12        uwEvent = LOS_EventRead(&EventGroup_CB, //事件标志组对象
13                                KEY1_EVENT|KEY2_EVENT, //等待 BIT0 和 BIT1
14                                LOS_WAITMODE_AND,
15                                LOS_WAIT_FOREVER ); //无期限等待
16        if((KEY1_EVENT|KEY2_EVENT) == uwEvent){
17            printf ( "KEY1 与 KEY2 都按下\n");
18            LED1_TOGGLE; //LED1 翻转
19            LOS_EventClear(&EventGroup_CB, ~KEY1_EVENT); //清除事件标志
20            LOS_EventClear(&EventGroup_CB, ~KEY2_EVENT); //清除事件标志
21        }
```

```
22     else{
23         printf ( "事件错误! \n");
24     }
29 }
30 }
```
